

Convolutional Neural Networks Part 1

Agenda

Motivation

Convolutional Layer

PyTorch Implementation

More on CNNs

How were the last four weeks?

Q: How is the pace of the lectures? Difficulty level?

Q: What do you think of the assignment? How's the difficulty level?

Q: How are the Q/A Sessions going?

We welcome your feedback. Please reach out if you have suggestions.

Recap: Motivating Problem -1

➤ Car vs Semi-Truck Classifier

Input: image



?

Output: pick the correct categories



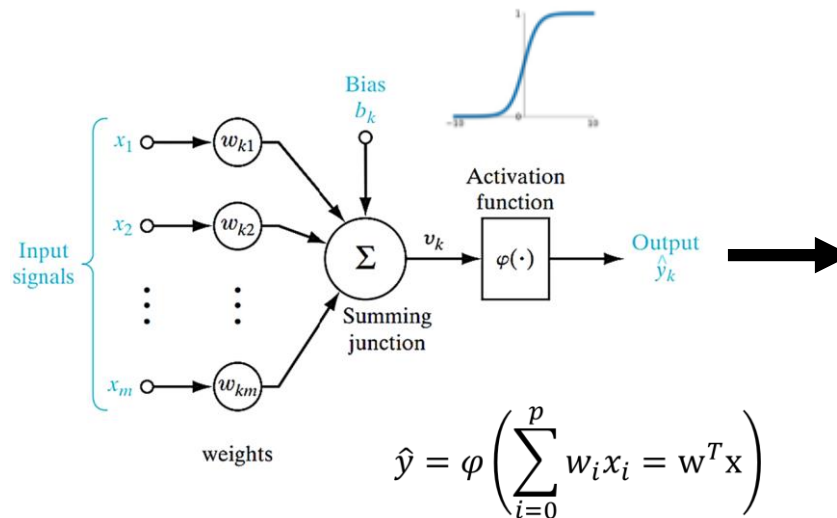
1 - car

0 - truck

Recap: Motivating Problem -2

1. Dataset: CIFAR
2. Model: Logistic Regression (simple linear network)

Input: image [1 x 1024]



Output: pick the correct categories

1 - car
0 - truck

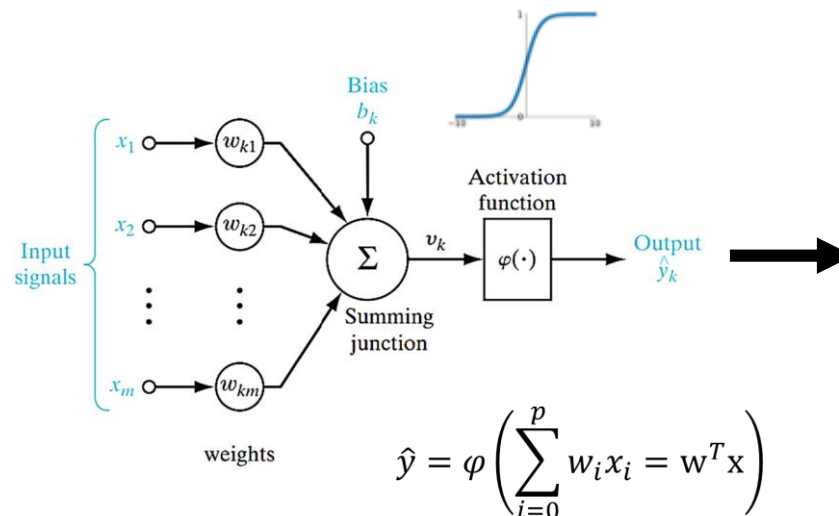
Accuracy: 65%

Recap: Motivating Problem -3

➤ Logistic Regression + Feature Engineering

Input:

[length, width, height, size] 1×4



Output: pick the correct categories

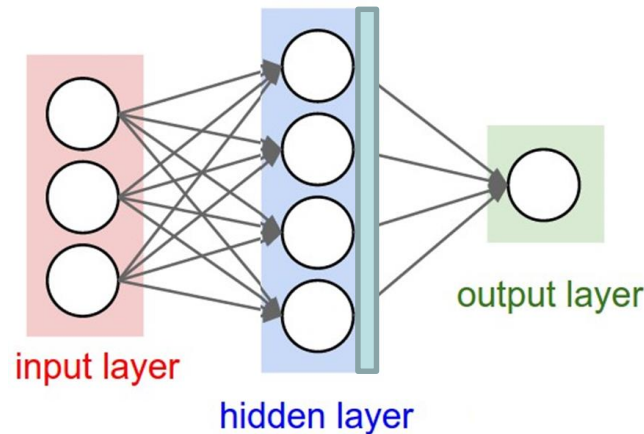
1 - car
0 - truck

Accuracy: 99%

Recap: Motivating Problem -4

- Multi-Layer Perceptron (MLP) to automate feature selection (i.e., nonlinear neural network)

Input: image [1 x 1024]



Output: pick the correct categories

1 - car
0 - truck

Accuracy: ~74%

Recap: Motivating Problem -4

➤ Q: Can we do better? And how?

Input: image



?

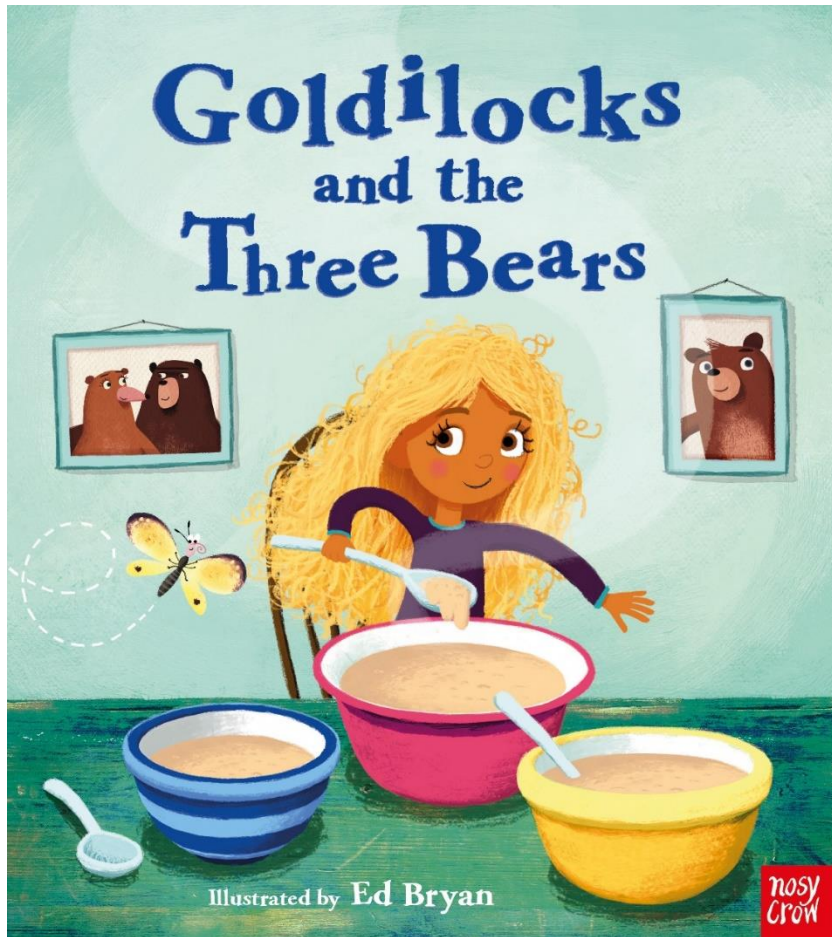


Output: pick the correct categories

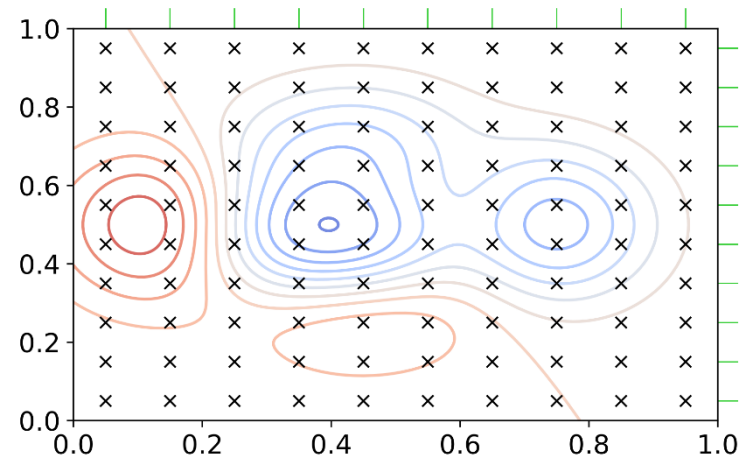
1 - car

0 - truck

1. Hyperparameter Tuning

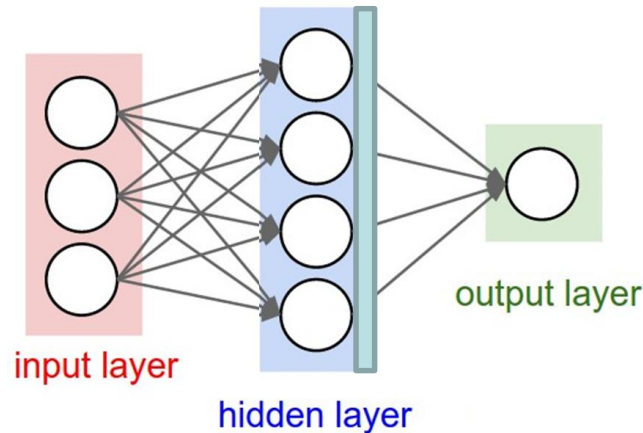


- Neural networks require a great deal of time tuning hyperparameters
- Practice makes perfect



2. Use Higher Resolution Images

Input: image [200 x 200]



Output: pick the correct categories

1 - car
0 - truck

- This will work, but there are some challenges to overcome...

PyTorch Example: # of Parameter

```
class Classifier(nn.Module):  
    def __init__(self):  
        super(Classifier, self).__init__()  
        self.layer1 = nn.Linear(32 * 32, 50)  
        self.layer2 = nn.Linear(50, 20)  
        self.layer3 = nn.Linear(20, 1)  
    def forward(self, img):  
        x = img.view(-1, 32 * 32)  
        x = F.relu(self.layer1(x))  
        x = F.relu(self.layer2(x))  
        x = self.layer3(x)  
        return x
```

Q: How many layers does this network have?

Q: How many weights are in the first layer of this network?

Q: How many total parameters are there?

High Resolution Image

200 pixels



200 pixels

What if our network is bigger?

Example:

- Input image: 200×200 pixels
- First hidden layer: 500 units
- Second hidden layer: 200 units

Q: How many parameters are there?

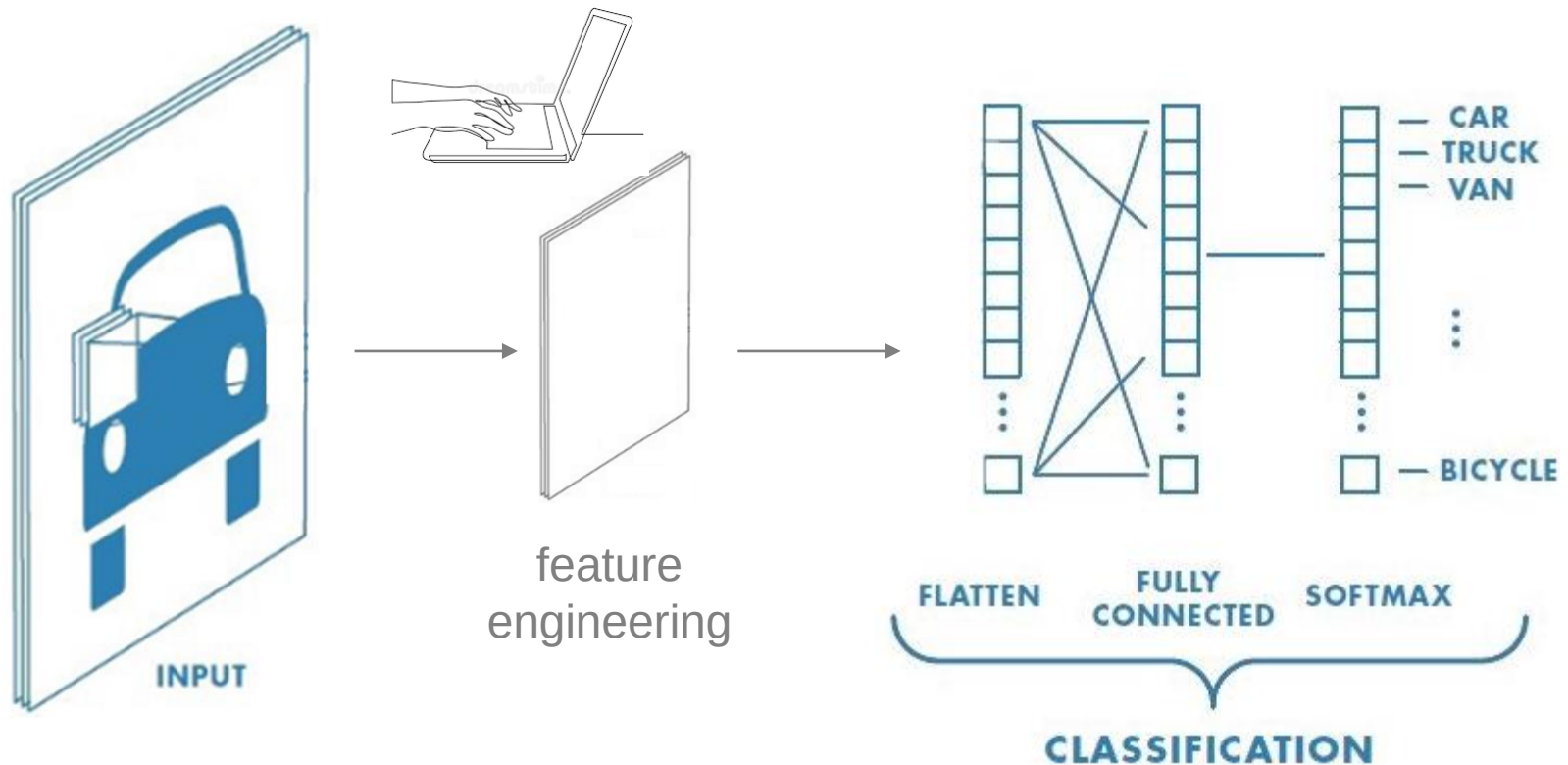
Q: Why would using a large fully connected layer be problematic?



What if our network is bigger?

- Q: Why would using a large fully connected layer be problematic?
 - slow training due to large number of parameters
 - requires a lot of training data to avoid overfitting (exponential relationship with # of parameters)
 - does not make use of spatial relationships
 - small shift in image can result in large change in prediction (each pixel would be multiplied by a different parameter/weight)

What was done in the past?



Most of the effort went into feature engineering or processing of the input.

Convolutional Filters

- Engineers have used the idea of **convolutional filters in computer vision** even before the rise of machine learning.
- They hand-coded filters that can detect simple features, for example to **detect edges** in various orientations.



Convolution in 2-D for Images

Convolution of Image **I** with filter kernel **K**:

- Flip rows and columns of kernel
- Multiply each pixel in range of kernel by the corresponding element of flipped kernel, sum all these products and write to a new 2d array.
- Slide kernel across all areas of the image until you reach the ends.
 - Optional, can use zero padding to maintain size of output

K

1	1	0
1	0	-1
0	-1	-1

I

1	2	2	2
0	1	4	3
0	2	2	2
0	1	1	0

Demonstration of 2D Convolution

kernel

-1	-1	0				
-1	0	1				
0	1	1	1	2	2	2
			0	1	4	3
			0	2	2	2
			0	1	1	0

input



1					

-1	-1	0				
-1	0	1				
0	1	1	1	2	2	2
			0	1	4	3
			0	2	2	2
			0	1	1	0



1	3				

...

1	-12	-12	0	2		
0	-11	0	4	1	3	
0	0	2	1	2	1	2
0	1	1	0			



1	3	4	4	2	0
1	3	6	7	1	-2
0	2	5	2		

...

1	2	2	2			
0	1	4	3			
0	2	2	2			
0	1	1	-10	-1	0	
			-1	0	1	
			0	1	1	



1	3	4	4	2	0
1	3	6	7	1	-2
0	2	5	2	-6	-5
0	3	3	-4	-9	-5
0	1	-1	-5	-5	-2
0	0	-1	-2	-1	0

output

Blurring Filter - Averaging

- Blurring averages out pixel intensities in an image.
- It is a form of low pass filter
- Attenuates high frequency features resulting from sharp transitions in the image

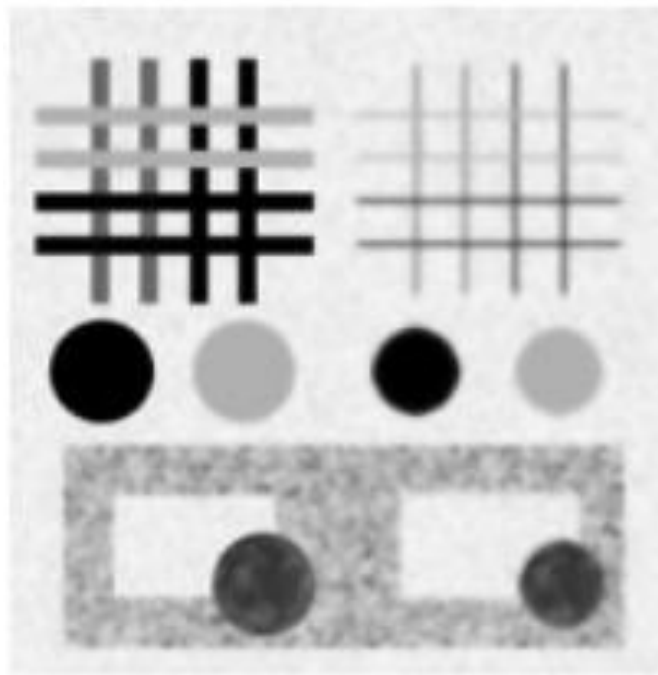
kernel

$\frac{1}{9}$	1	1	1
	1	1	1
	1	1	1



Sobel Filter X

Q: What does this filter do?

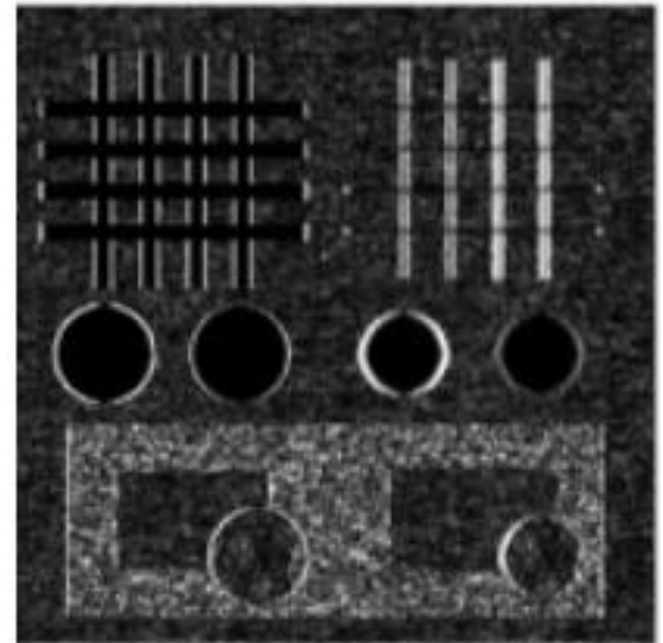


Original

kernel

-1	0	1
-2	0	2
-1	0	1

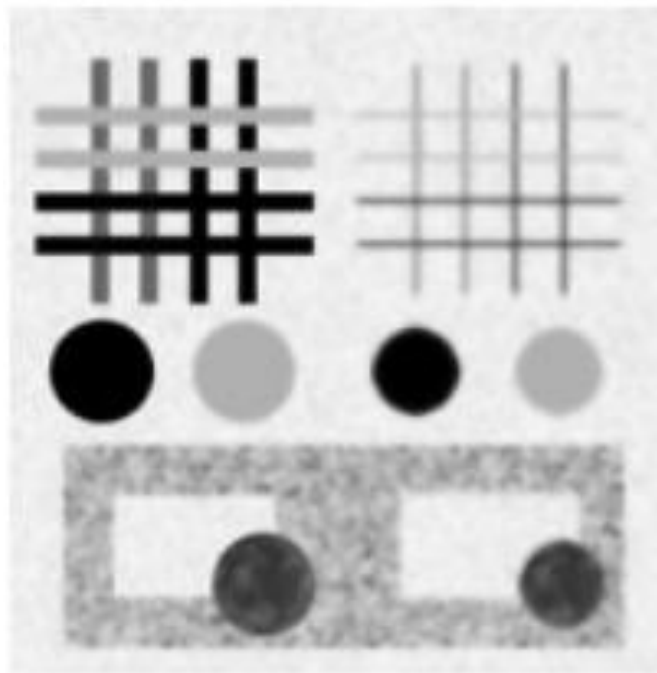
$*$ $=$



Sobel X

Sobel Filter Y

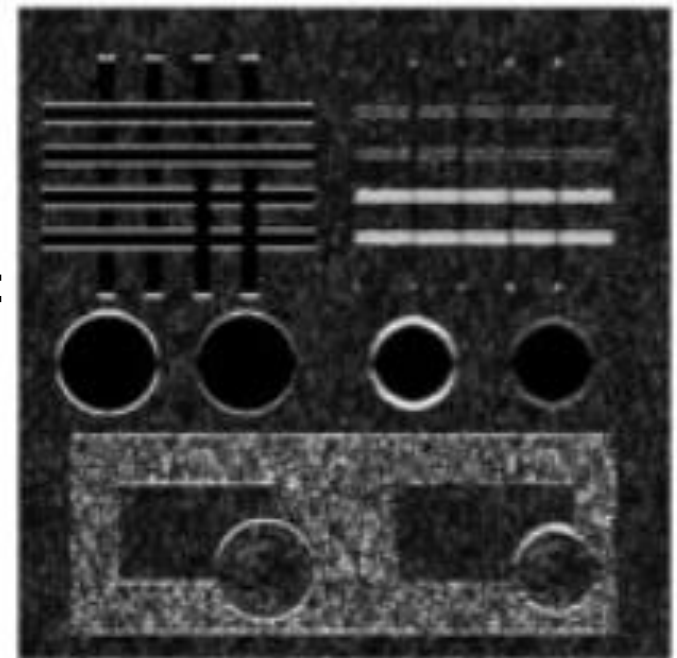
Q: What does the above filter do?



Original

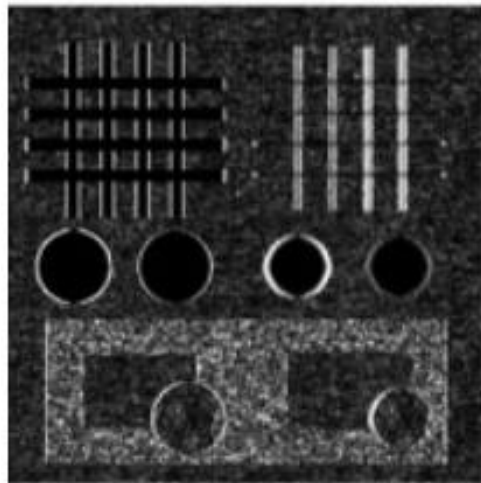
kernel

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array} =$$

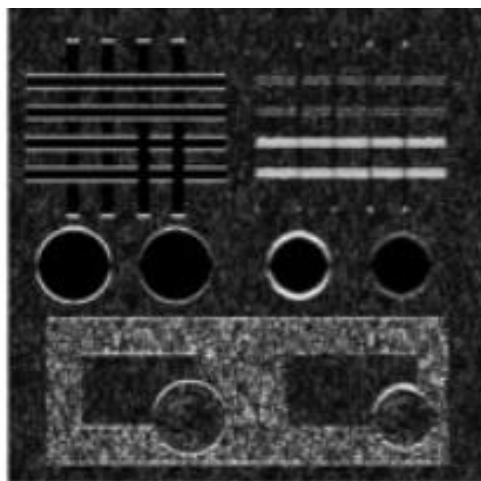


Sobel Y

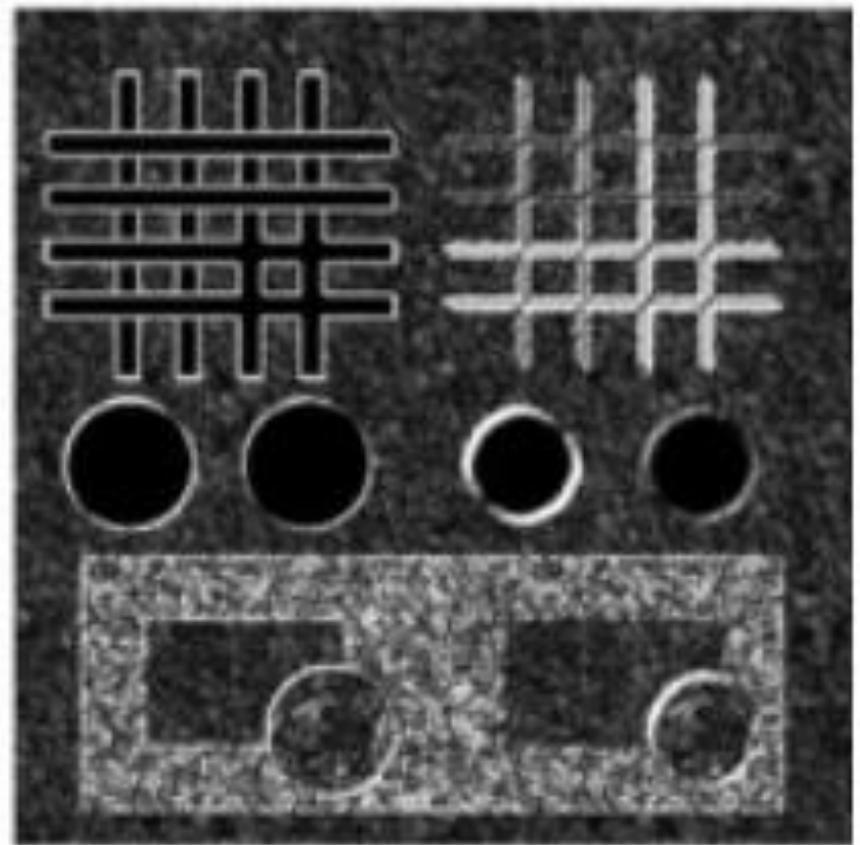
Horizontal and Vertical Combined



Sobel X



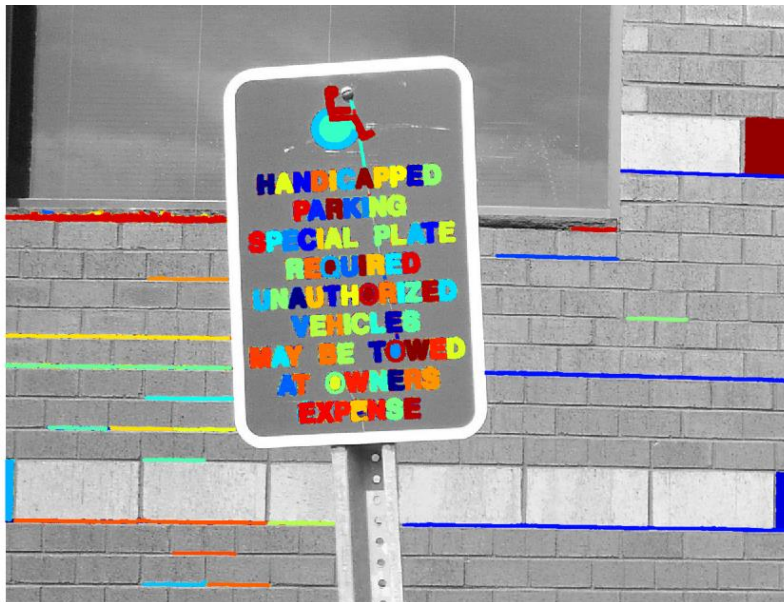
Sobel Y



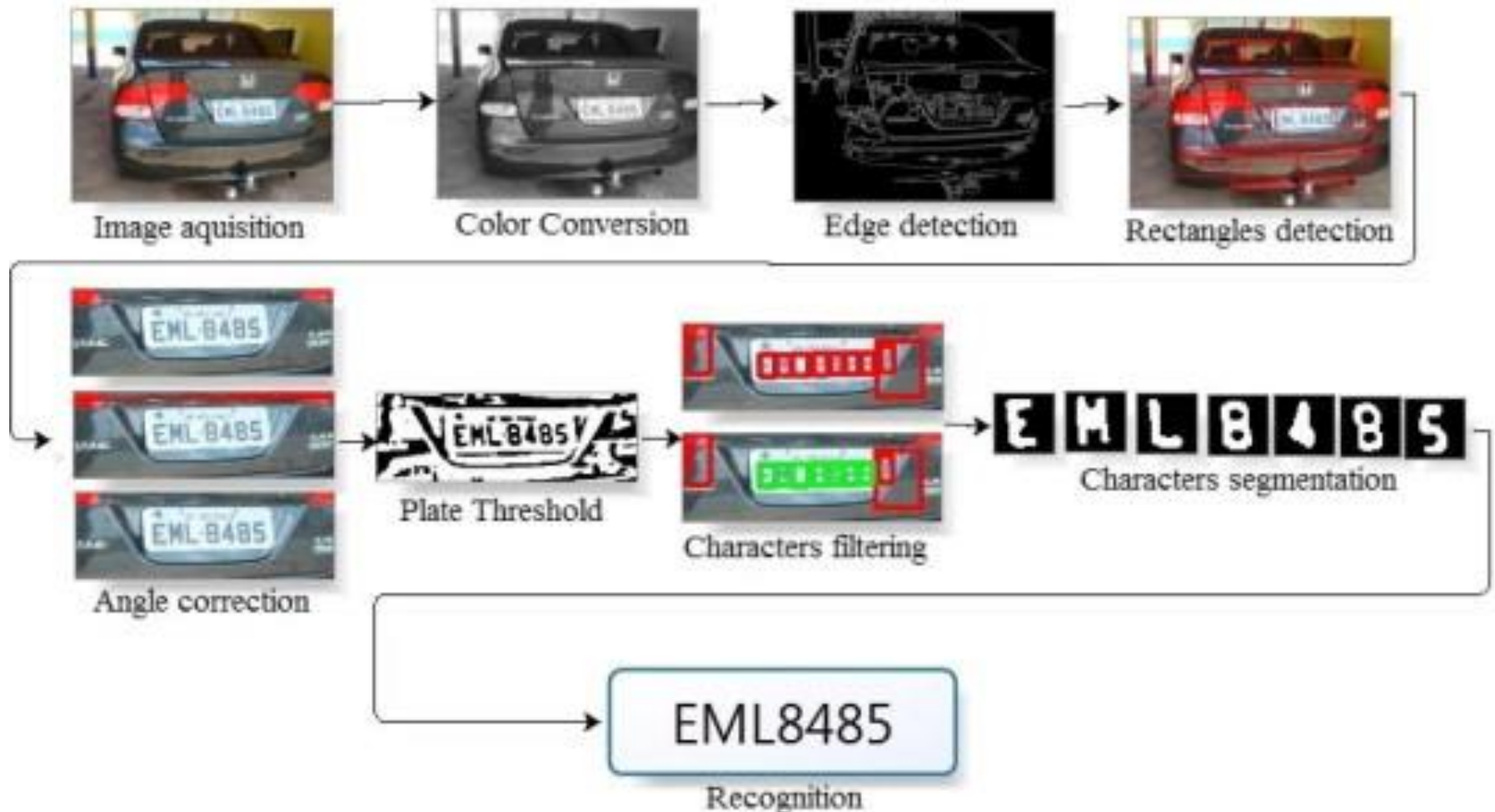
Sobel X+Y

Optical Character Recognition

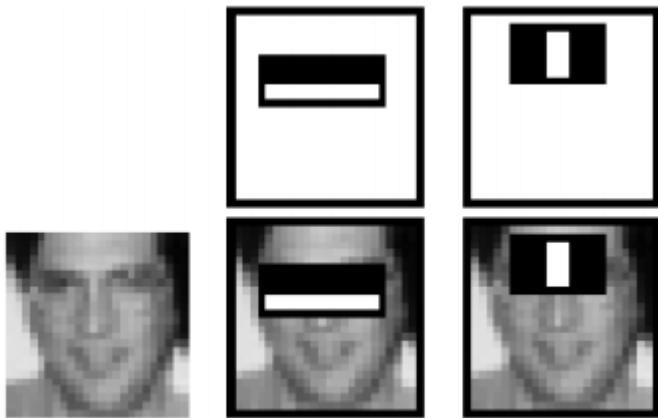
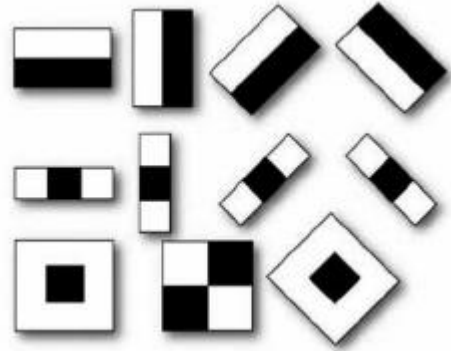
- Applies different filters and knowledge of characters to identify regions of interest
- Then eliminates regions one by one based on thresholds and line stroke characteristics



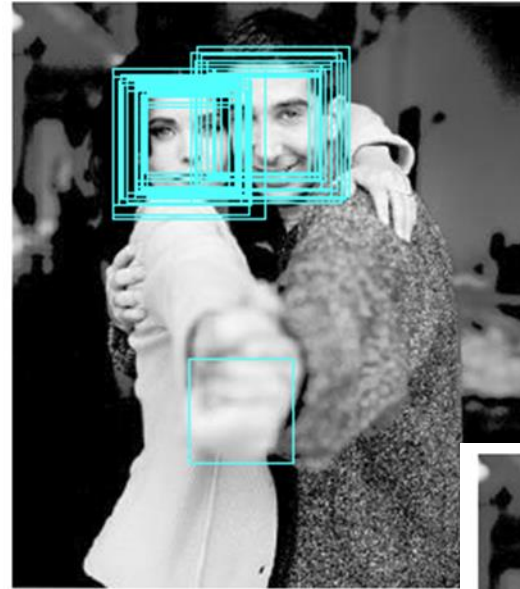
Optical Character Recognition



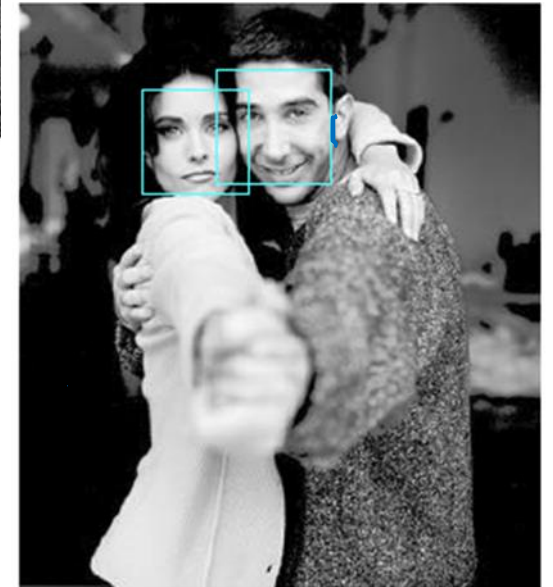
Face Detection



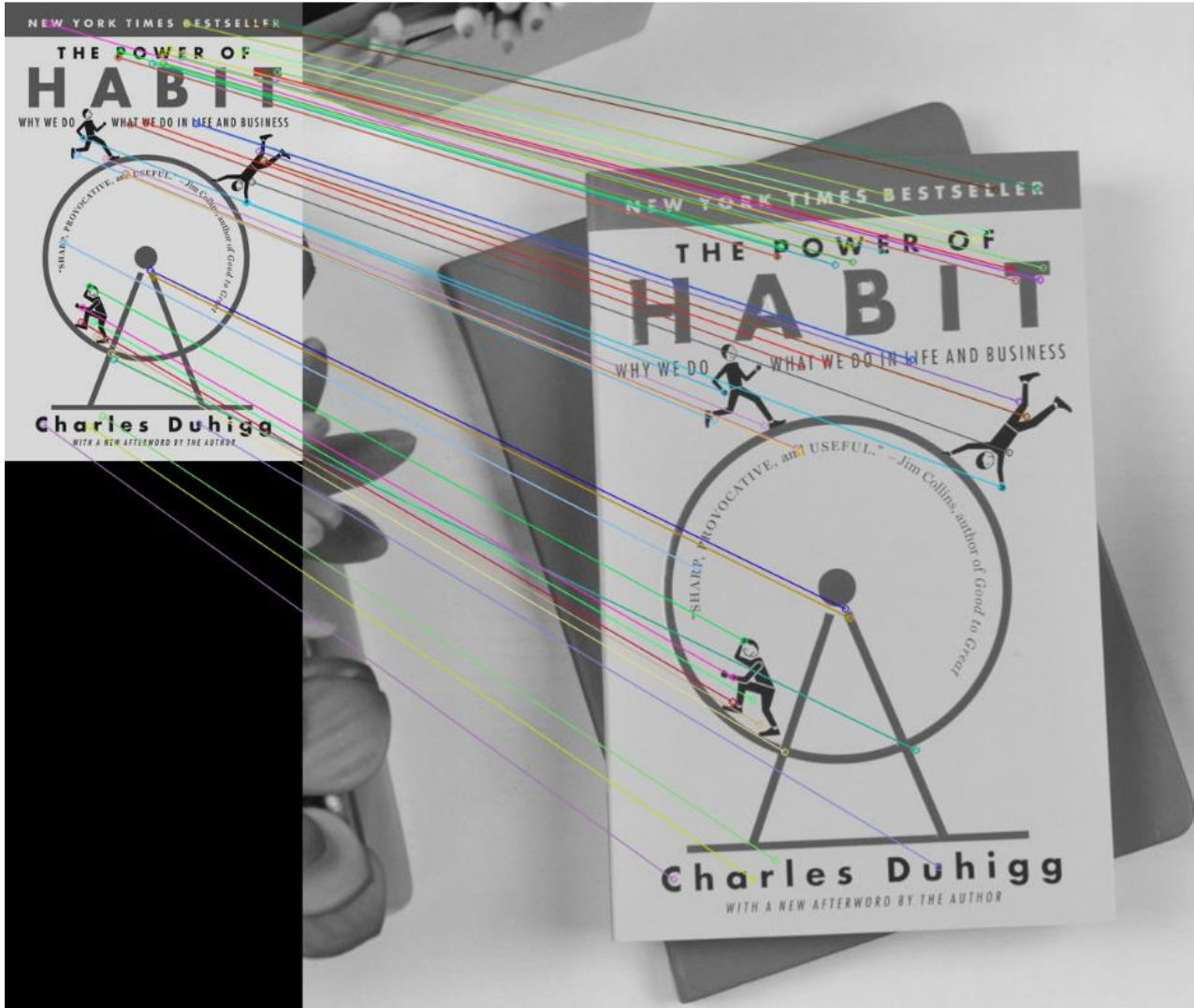
Convolutional kernels



➤ Viola-Jones algorithm achieved state-of-the-art results



Scale-Invariant Feature Transform (SIFT)



➤ Extract gradient information to find keypoints and descriptors (i.e., features) invariant to transformations

- Translation
- Rotation
- Scaling

Convolutional Neural Networks

Idea:

- Empower our neural networks with the ability to use convolution operations.

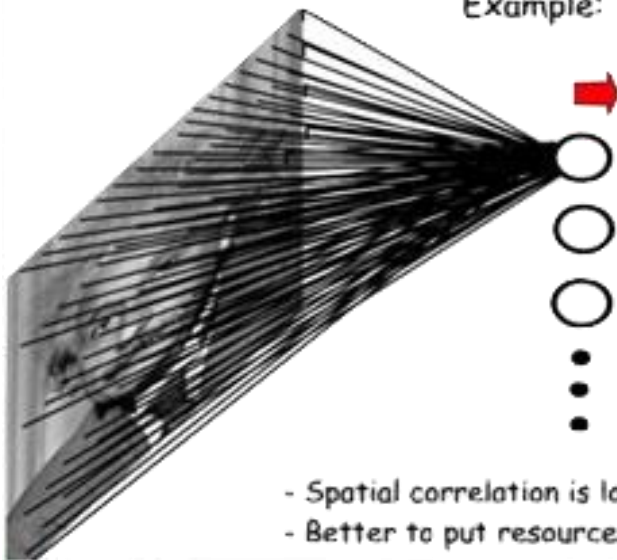
Key Points:

1. **locally-connected layers:** look for local (spatial) features in small regions of the image
2. **weight-sharing:** reuse the same weights across local regions over the entire image
3. **automate feature engineering:** neural network learns the kernel values, which become our parameters/weights

Requires Less Parameters

FULLY CONNECTED NEURAL NET

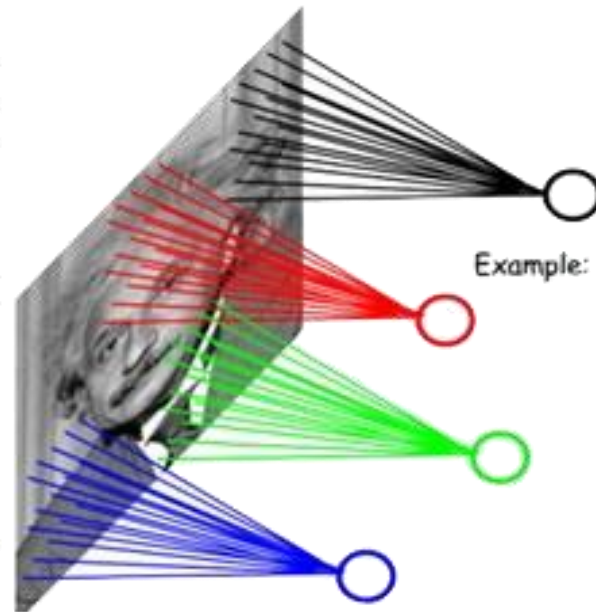
Example: 1000x1000 image
1M hidden units
→ 10¹² parameters!!!



- Spatial correlation is local
- Better to put resources elsewhere!

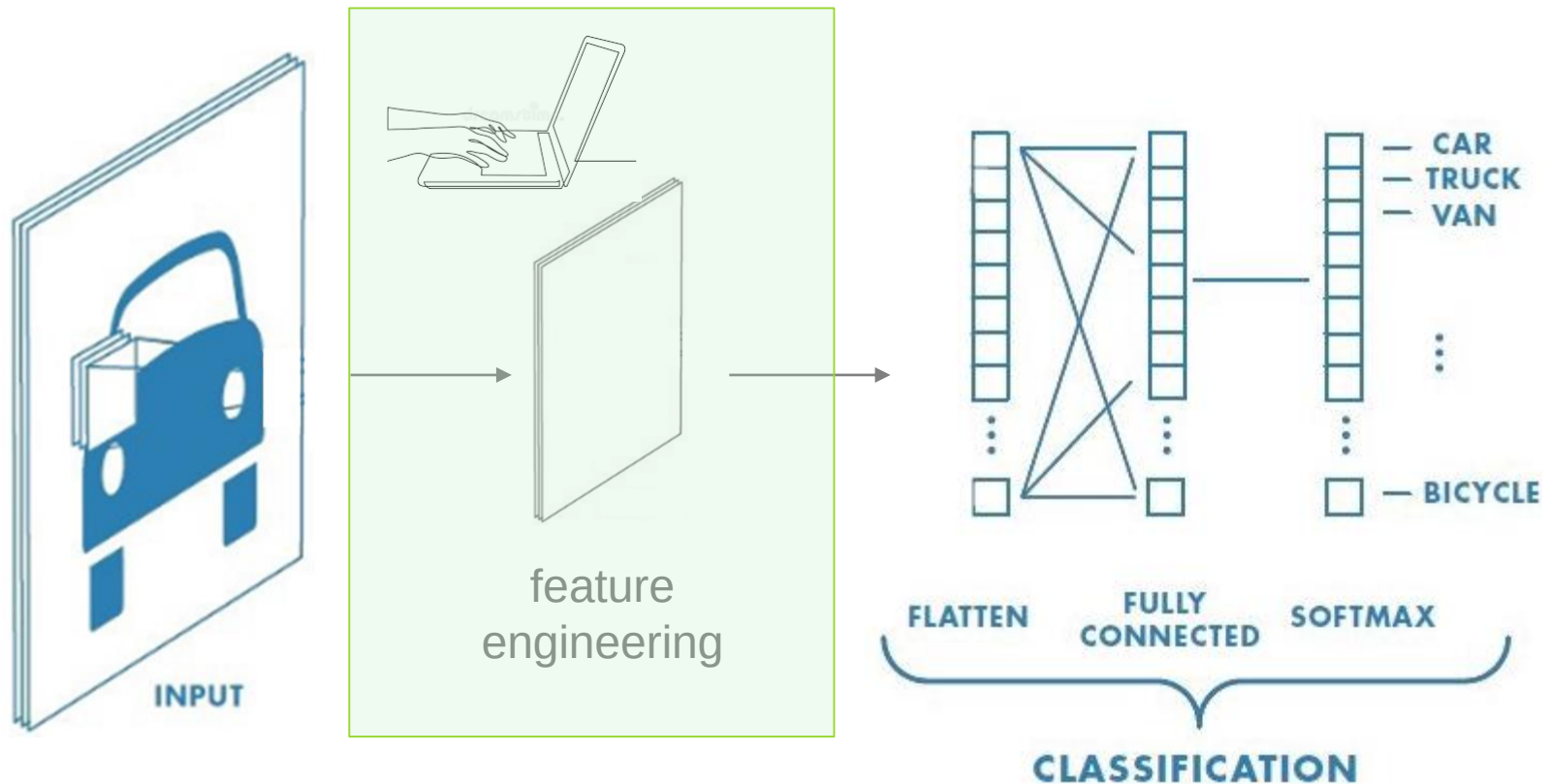
LOCALLY CONNECTED NEURAL NET

Example: 1000x1000 image
1M hidden units
Filter size: 10x10
100M parameters



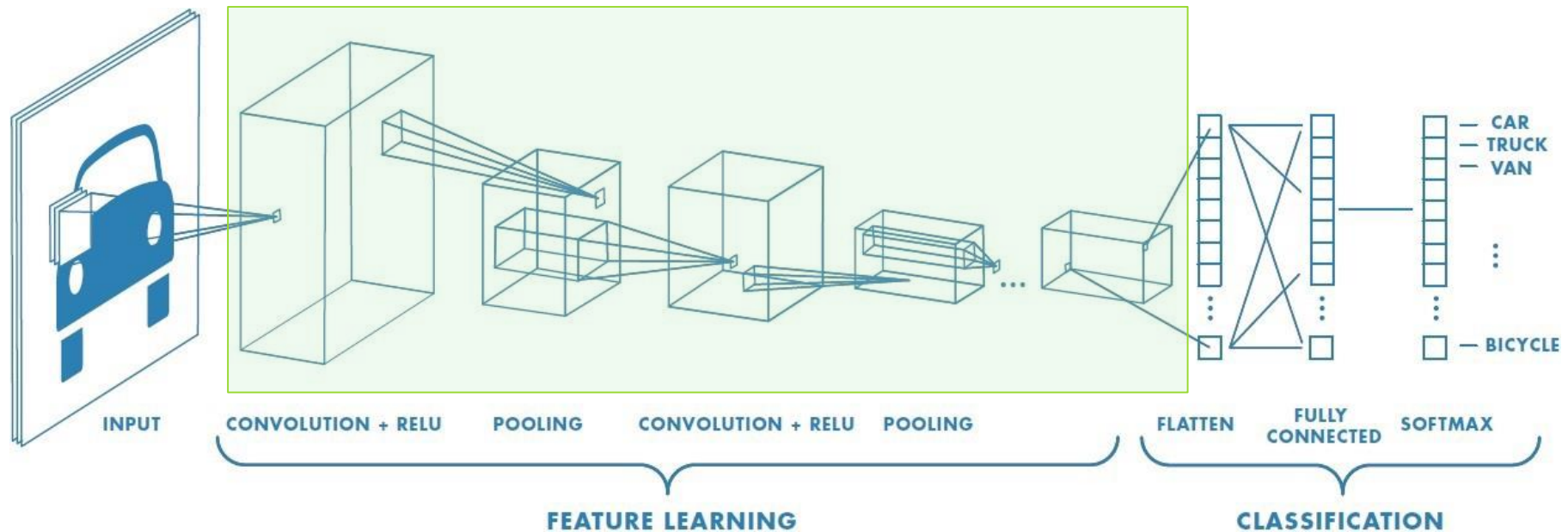
Convolutional Layers

Traditional Machine Learning



Most of the effort went into feature engineering or processing of the input.

Deep Learning



Automate the feature engineering process

Architecture **inspired by the organization of neurons in the visual cortex**



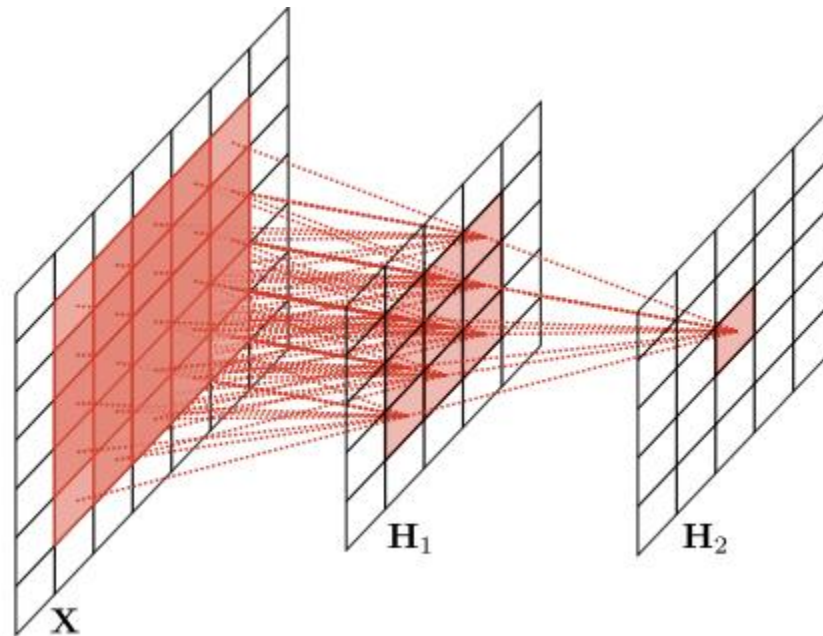
Biological Influence

Hubel and Wiesel Cat Experiments (1958 – 1959):

- Individual neurons respond to stimuli only in a restricted region of the visual field known as the **Receptive Field**.
 - collection of such fields overlap to cover the entire visual area.
 - some neurons react only to images of horizontal lines, while others react to line orientations
 - two neurons may have the same receptive field but react differently
 - higher-level neuron inputs are the outputs of neighbouring lower-level neurons

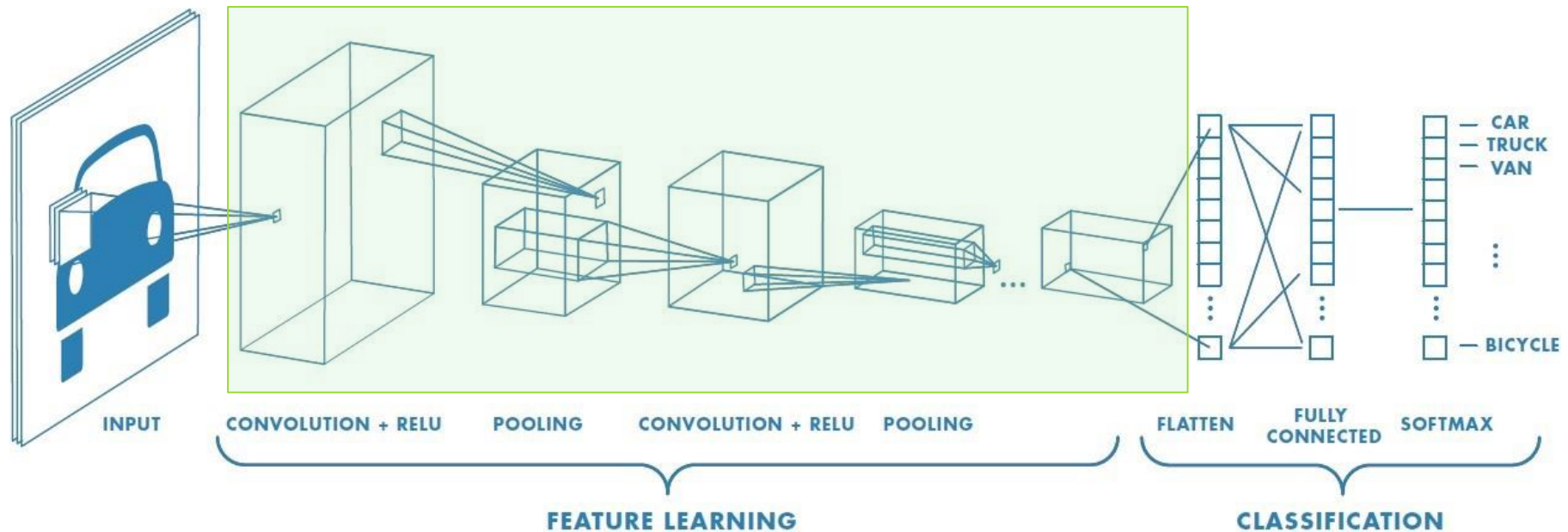
Receptive Fields

Refers to all the pixels in the input which influence the neuron being examined.



example receptive field of neuron H_2

Architecture: CNN Layers



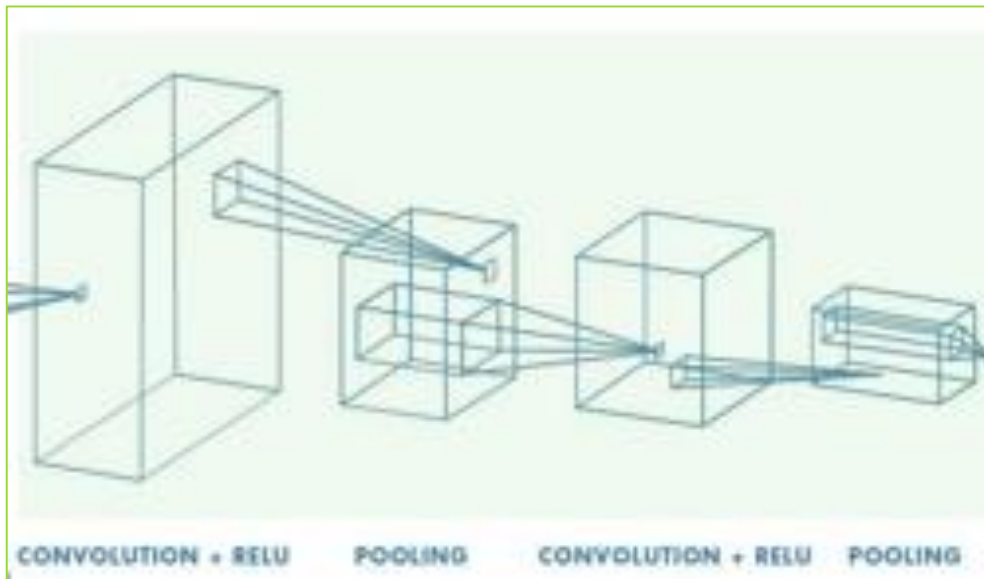
Introduce two new types of layers:

1. Convolutional layers
2. Pooling layers

Q: What is the geometry of the convolution and pooling layers?

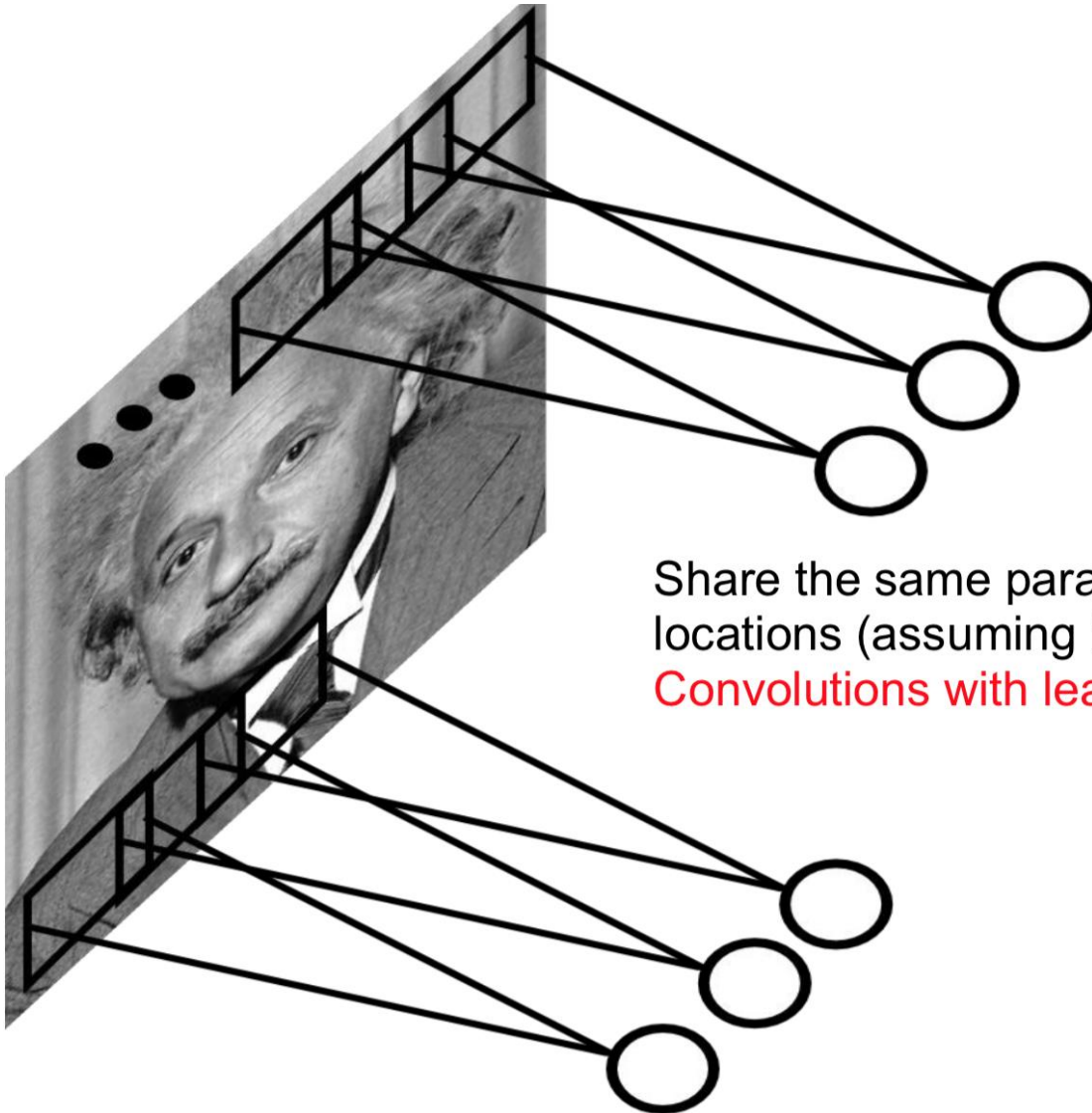
Locally Connected Layers

- **Notion of proximity:** To find out what object is at a particular pixel, can look at other pixels nearby.
 - nearby pixels will be more informative than pixels further away



maintains geometry
(top-right corner of the
hidden layers being
computed from the
top-right region of the
original input image)

Weight Sharing

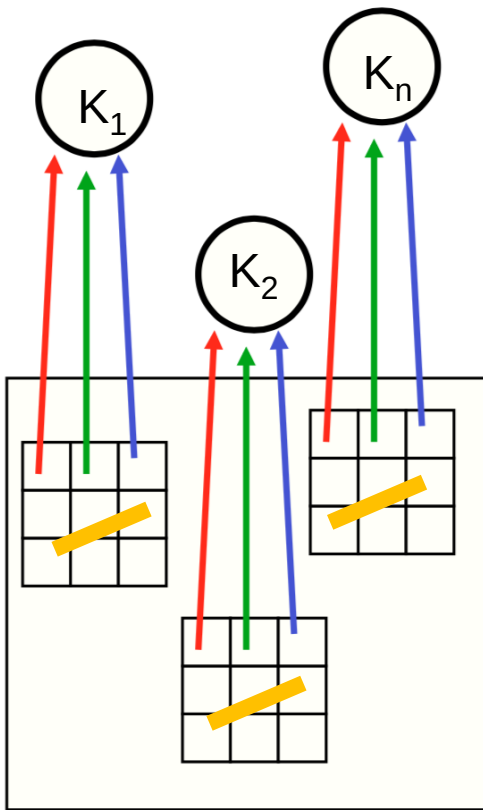


Share the same parameters across different locations (assuming input is stationary):

Convolutions with learned kernels

Weight Sharing

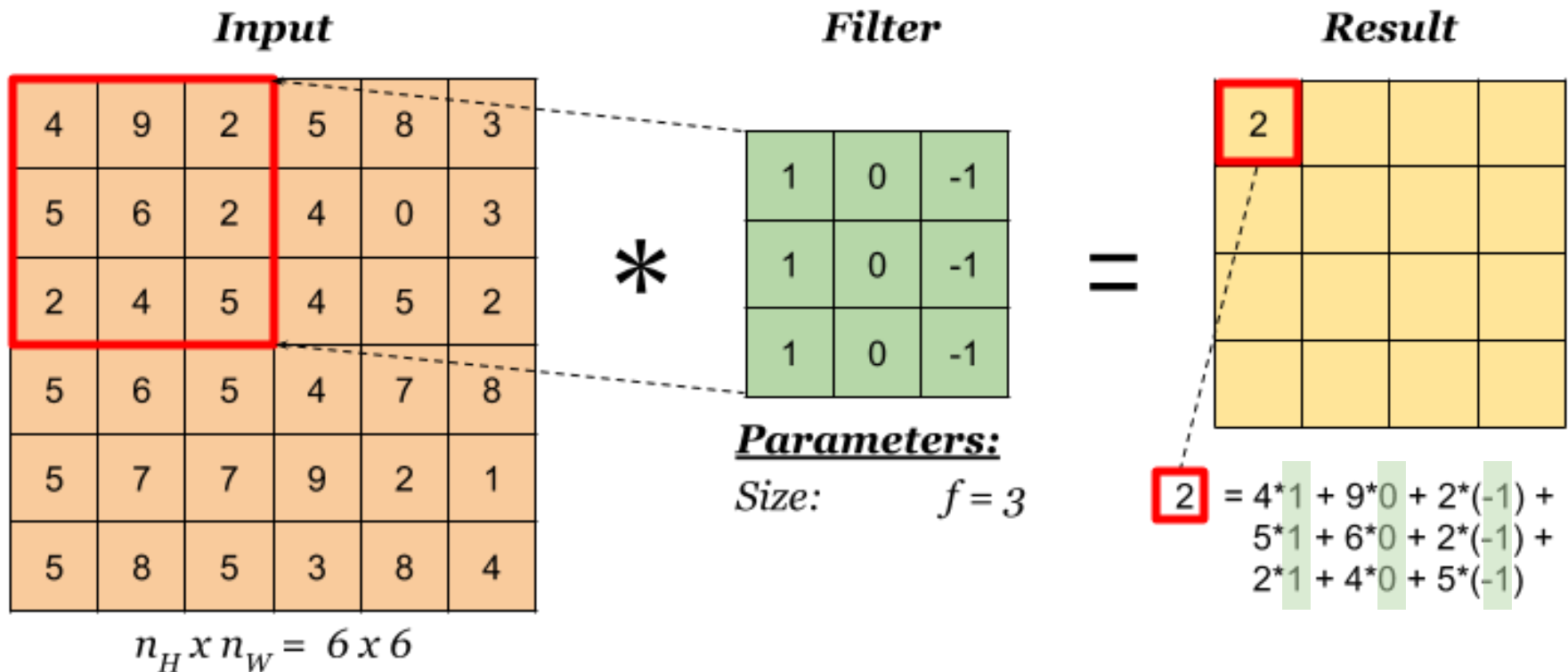
Each neuron on the higher layer is detecting the same feature, but in different locations on the lower layer



- “Detecting” = the output (activation) is high if the feature is present
- “Feature” = something in the image, like an edge, blob or shape

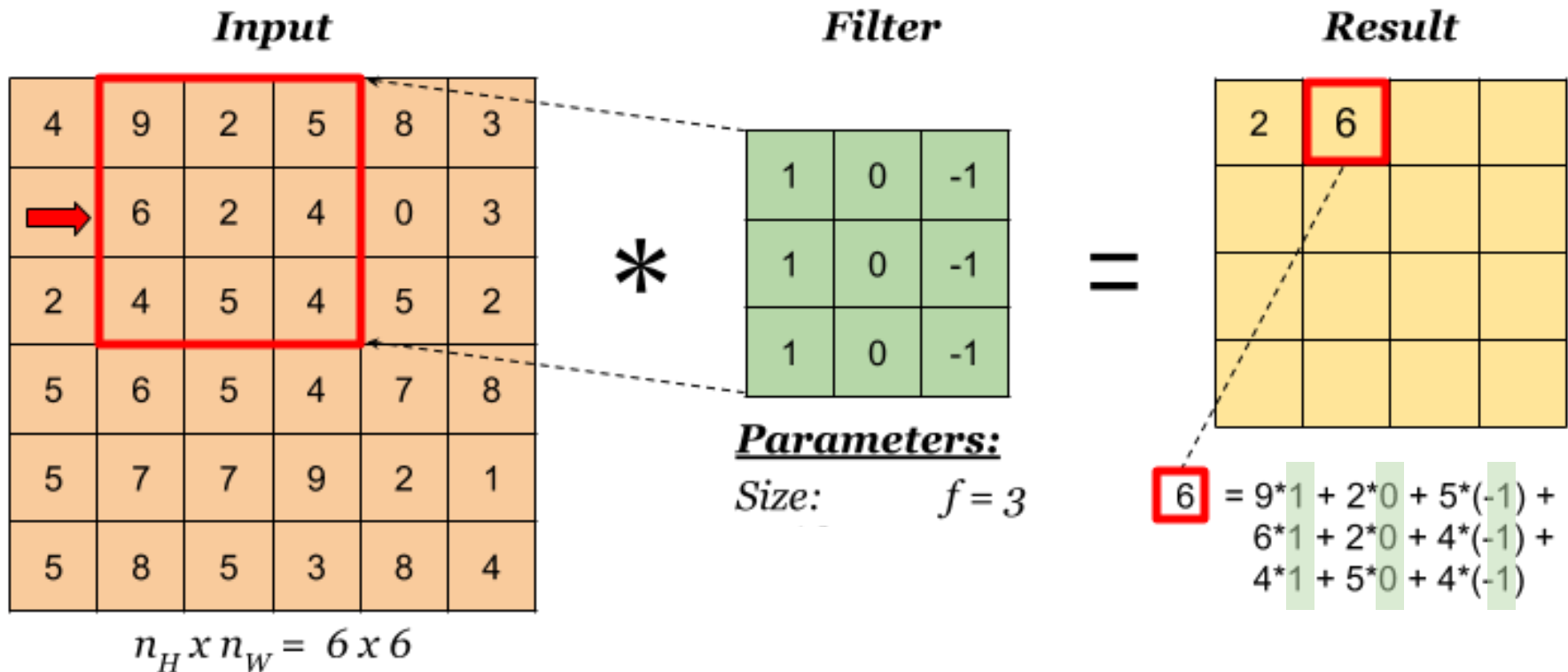
Convolution Layer Forward Pass

To compute the output, we superimpose the kernel on a region of the image.



Convolution Layer Forward Pass Con't

Slide one to the right and compute



Example Calculation - Feature Map

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



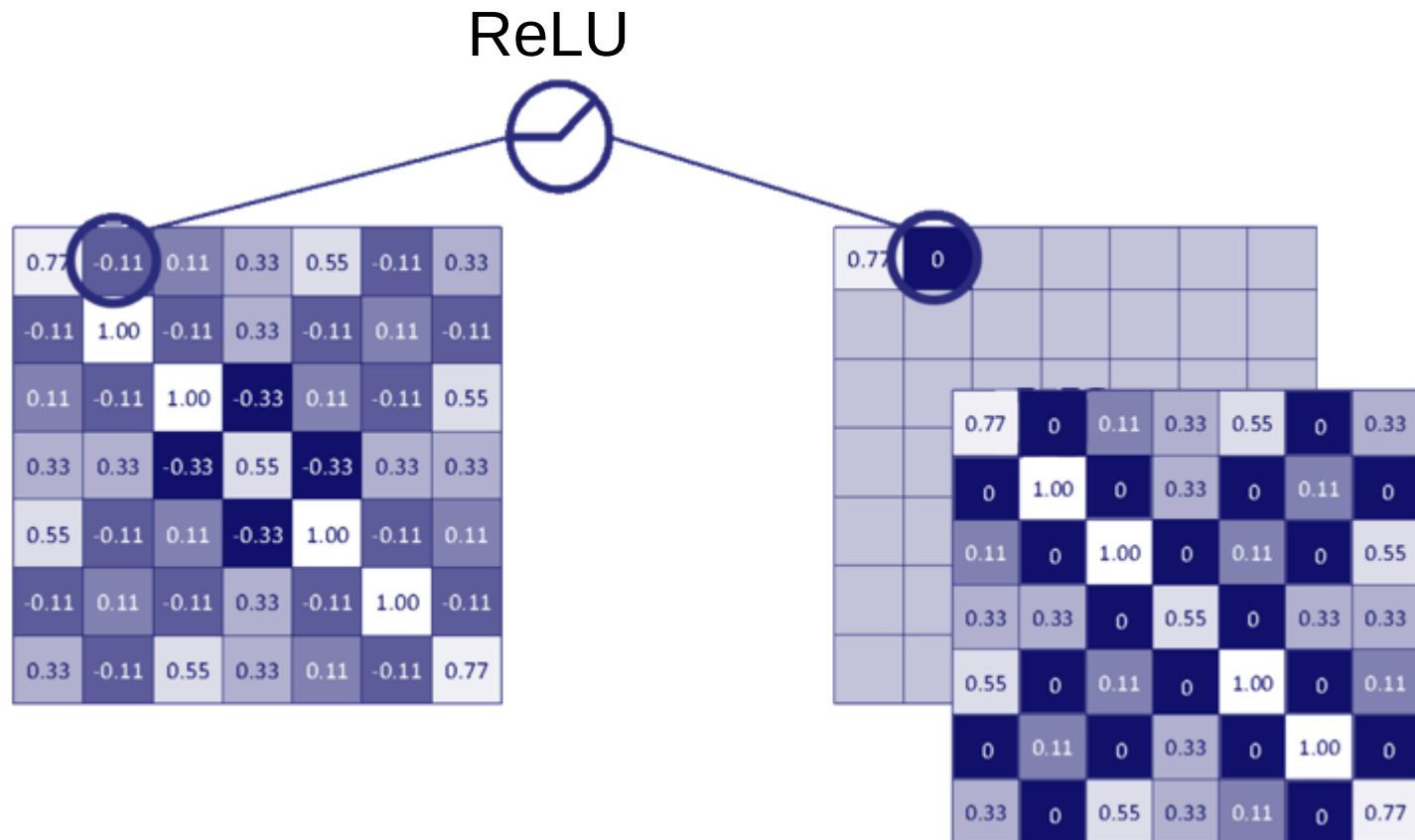
1	-1	-1
-1	1	-1
-1	-1	1

=

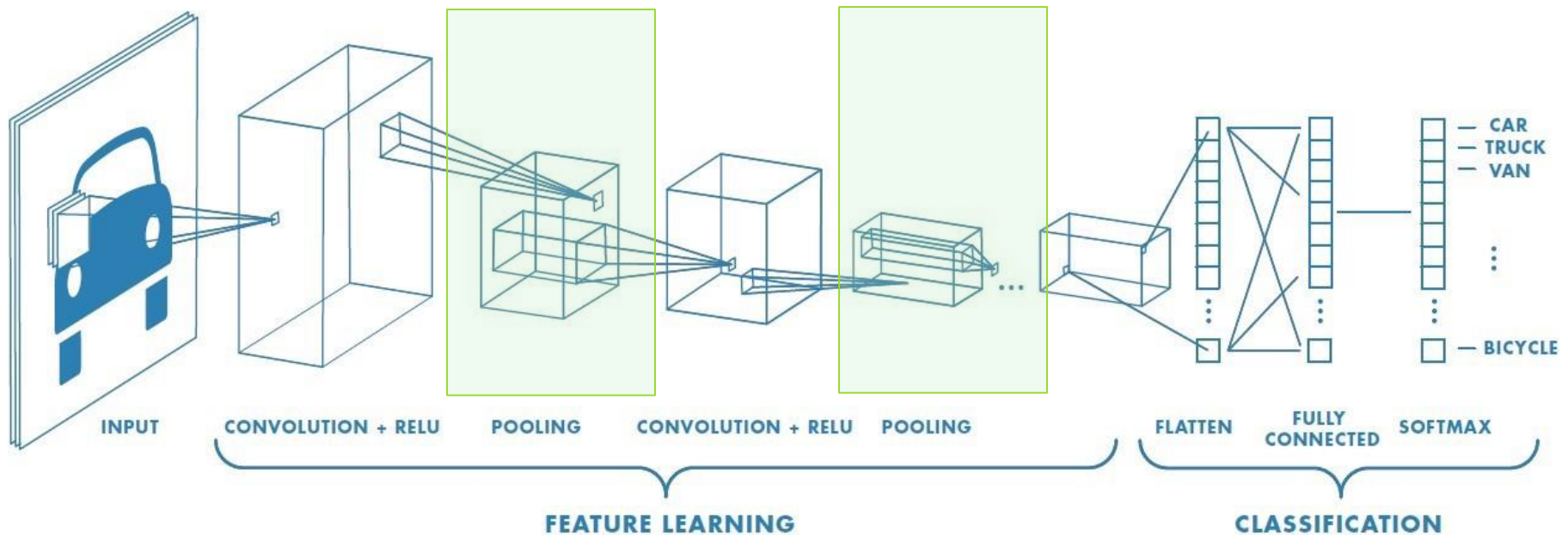
Output (**Feature Map**)

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

Example Calculation - Activation



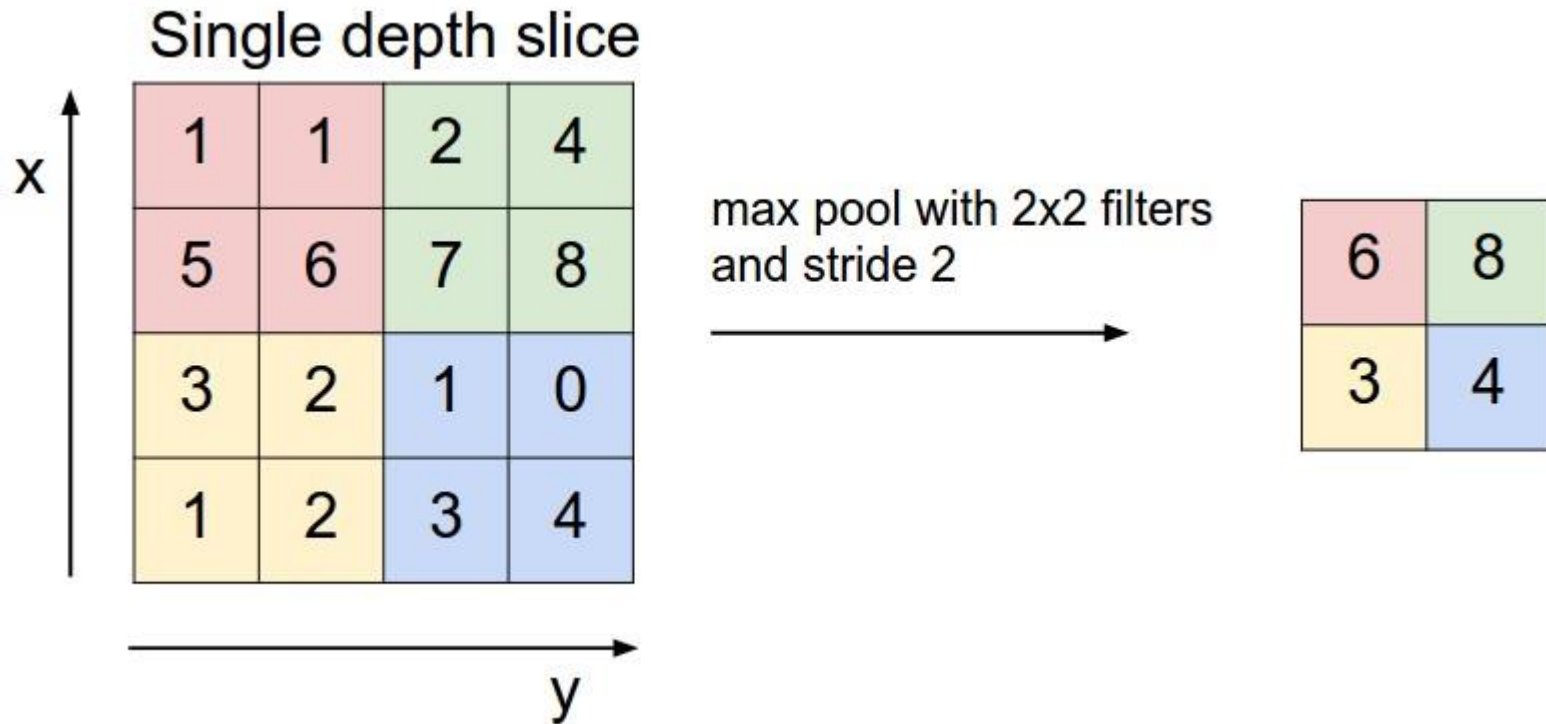
Pooling Layers



Pooling is a technique used to consolidate information.

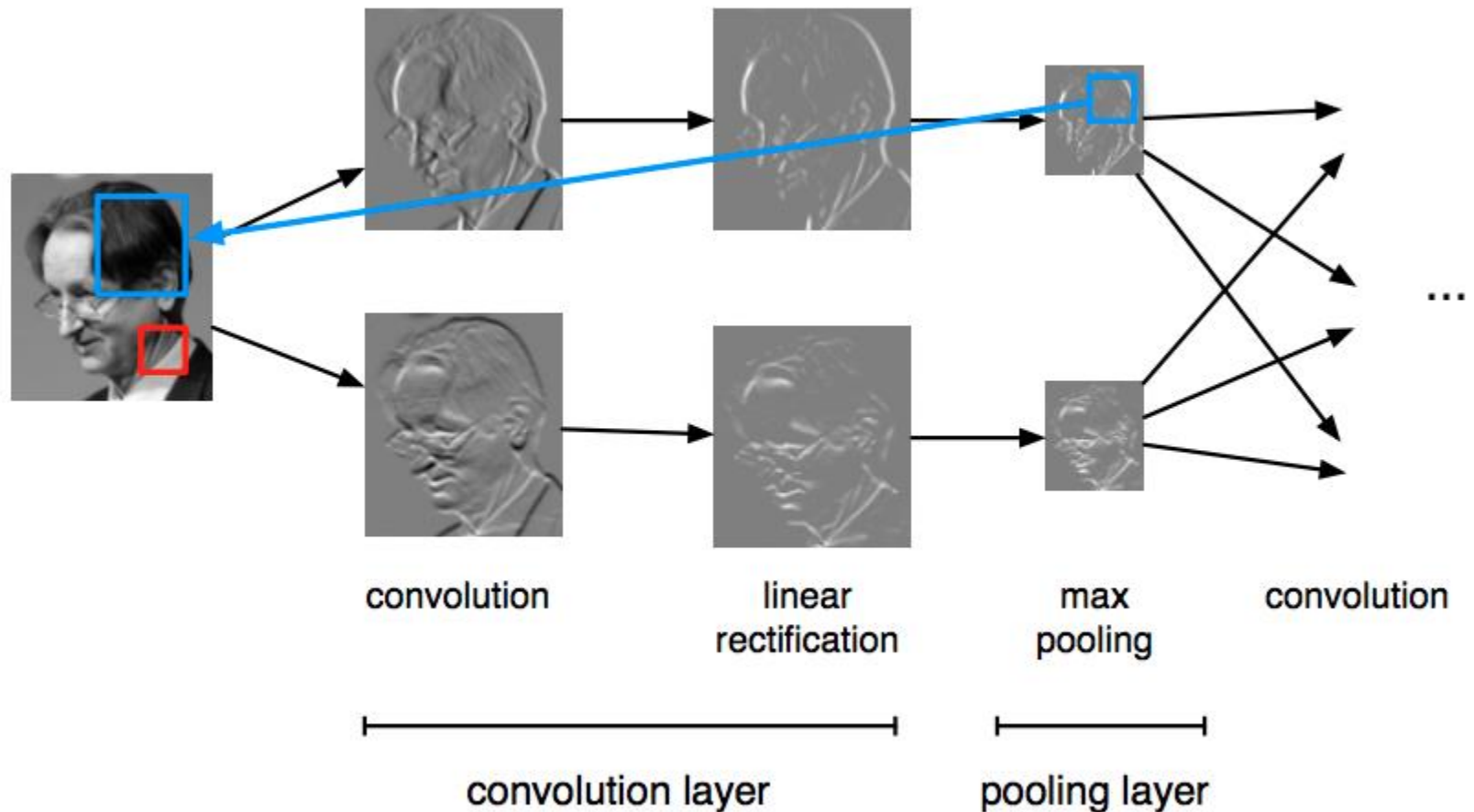
Max-Pooling

Idea: take the maximum value in each 2×2 grid



Max-Pooling Example

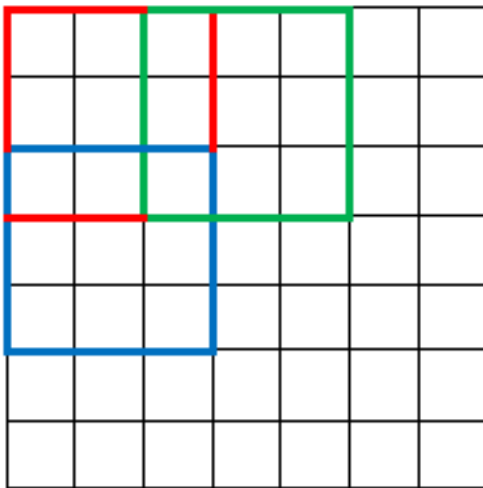
We can add a max-pooling layer after each convolutional layer



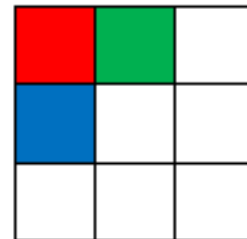
Strided Convolution

- Pooling operations less common now and generally replaced with strided convolutions

7 x 7 Input Volume



3 x 3 Output Volume



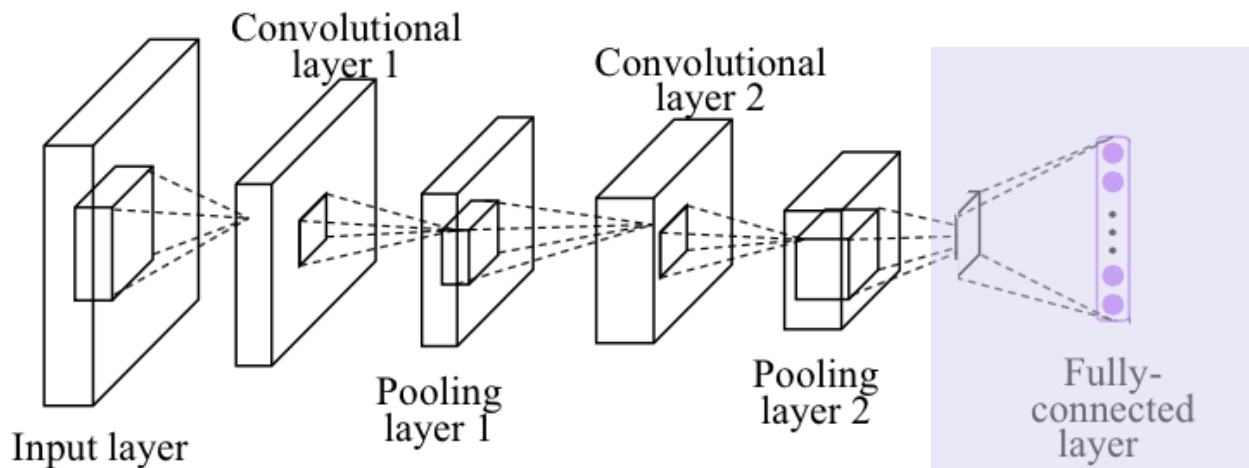
Shift the kernel by 2 (stride=2) when computing the next output feature.

PyTorch Implementation

CNNs in PyTorch: Linear Layers

Let us make a simple CNN in PyTorch

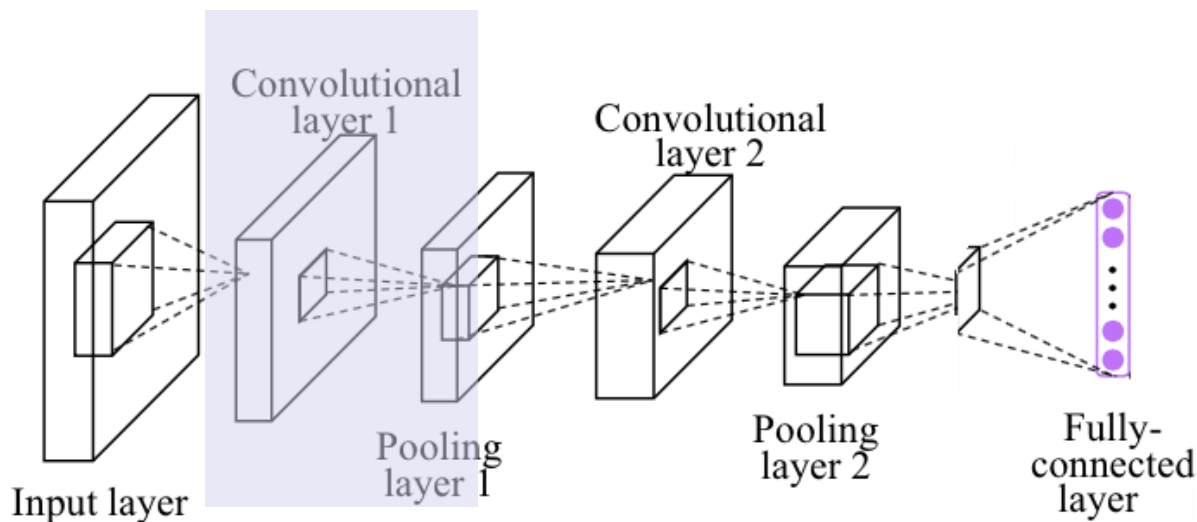
- Recall that in PyTorch, we can create a fully-connected linear layer between successive layers like this



```
self.fc = nn.Linear(n, 10)
```


CNNs in PyTorch: Convolutional Layers

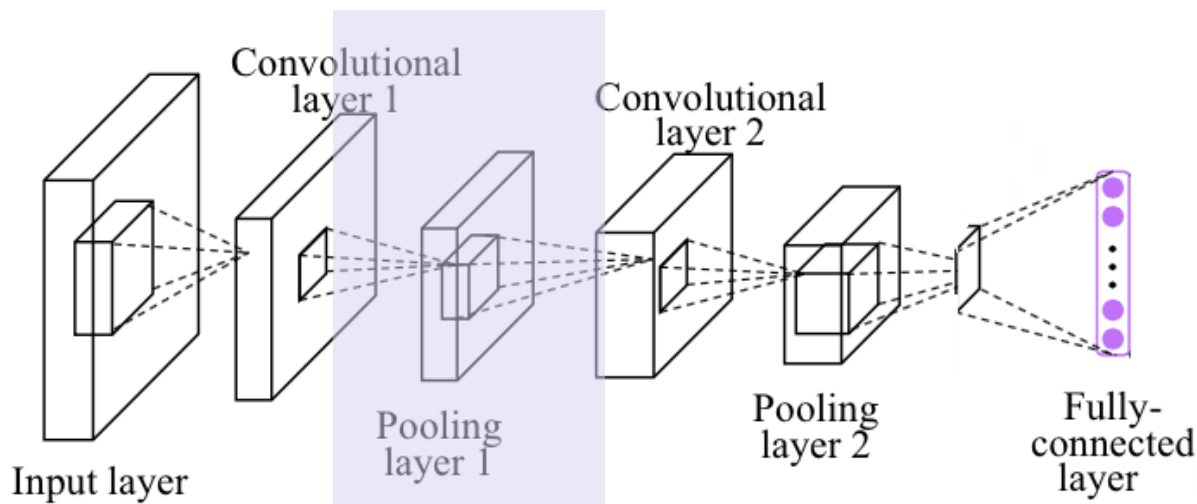
In PyTorch, we can create a convolutional layer using `nn.Conv2d`:



```
self.conv1 = nn.Conv2d(in_channels = 3,  
                        out_channels = 7,  
                        kernel_size= 5,  
                        stride = 1,  
                        padding = 1)
```

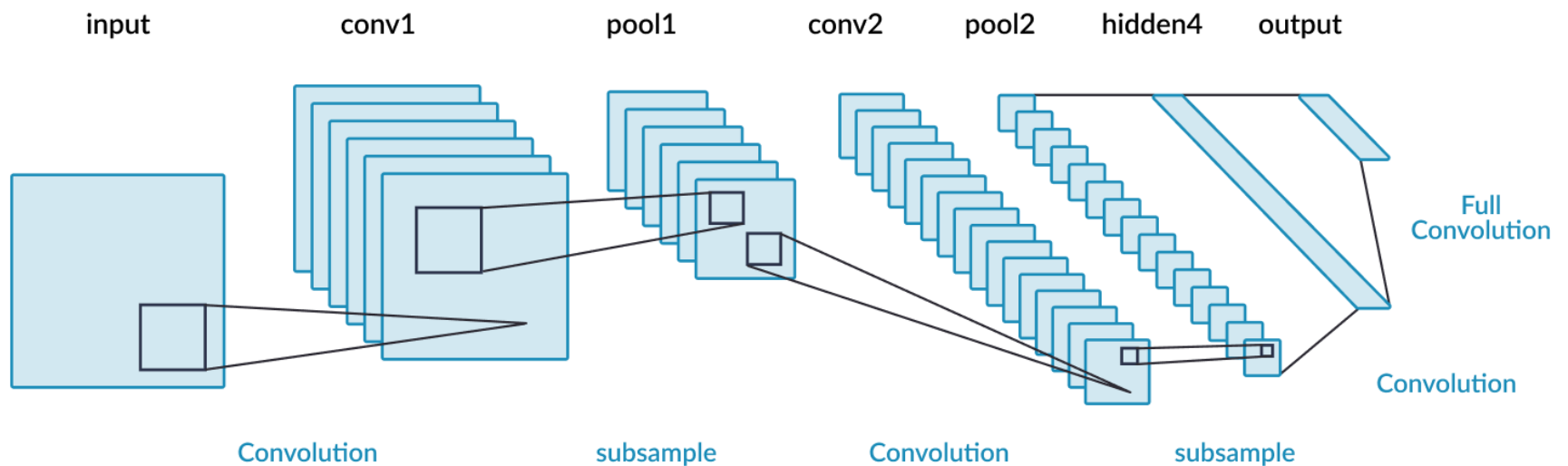
CNNs in PyTorch: Max Pooling Layers

In PyTorch, we can create a max pooling layer using `nn.MaxPool2d`:



```
self.pool = nn.MaxPool2d(kernel_size = 2,  
                           stride = 2)
```

CNN Architecture



Consolidation of information!



PyTorch Example CNN Architecture:

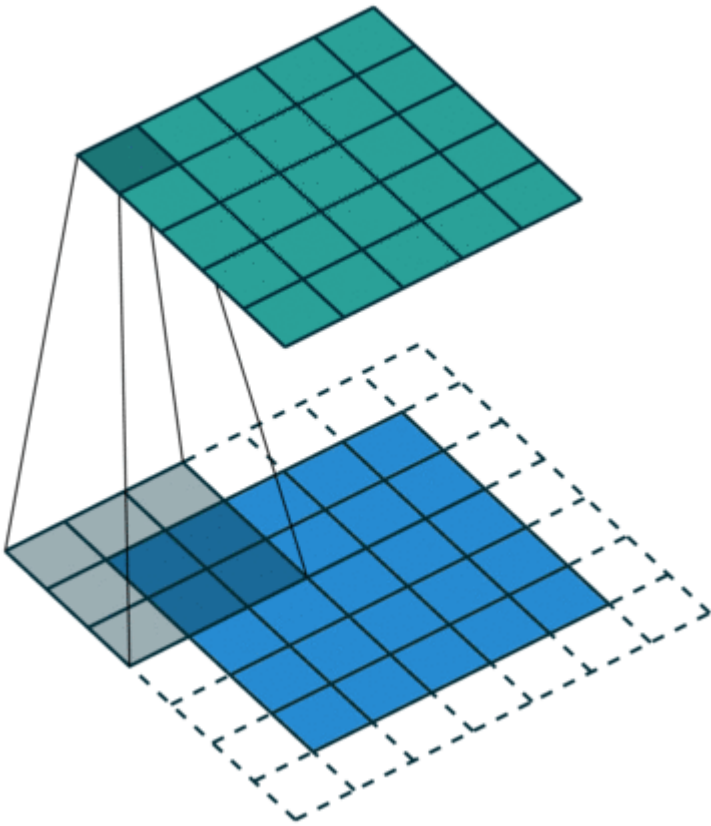
```
class LargeNet(nn.Module):
    def __init__(self):
        super(LargeNet, self).__init__()
        self.name = "large"
        self.conv1 = nn.Conv2d(3, 5, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(5, 10, 5)
        self.fc1 = nn.Linear(10 * 5 * 5, 32)
        self.fc2 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 10 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

PyTorch Examples

More on CNNs

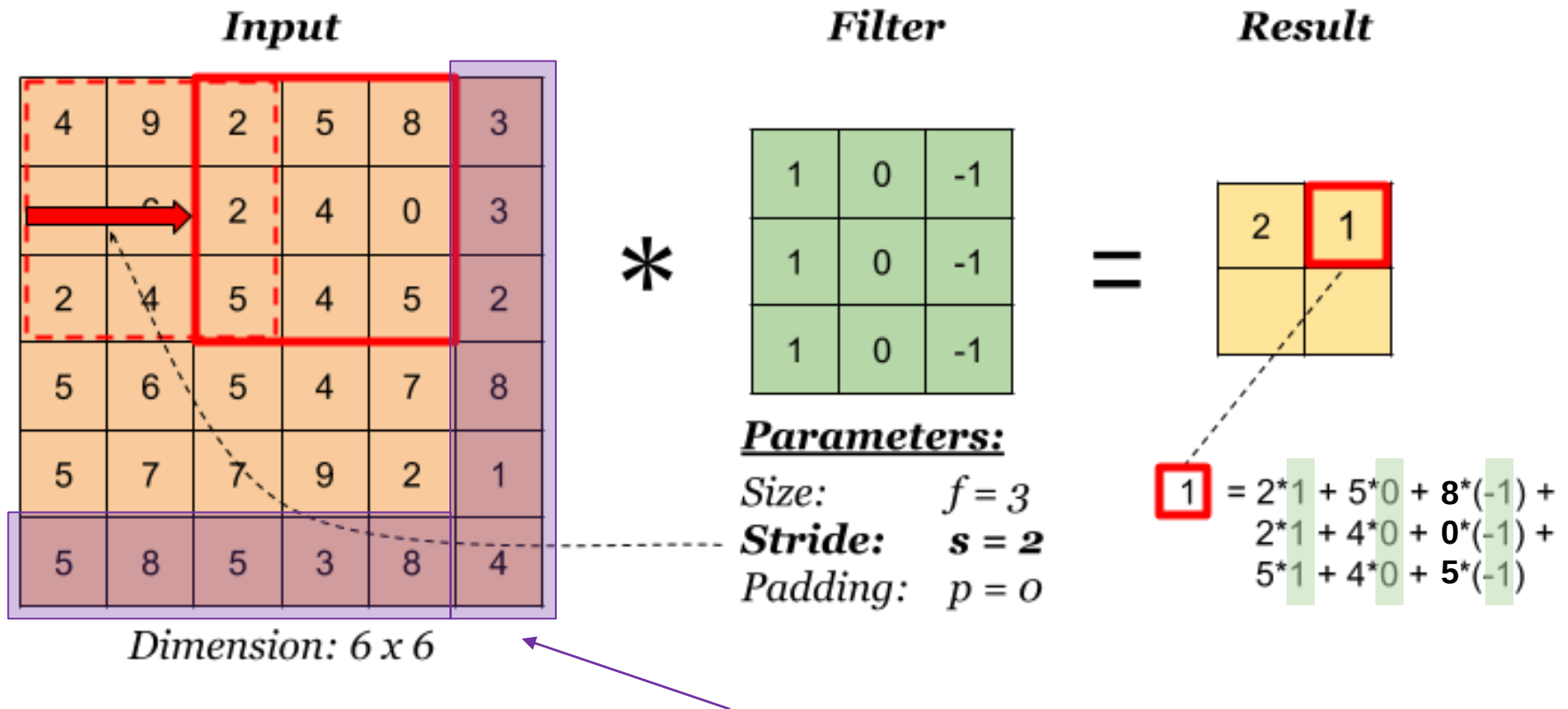
Zero Padding



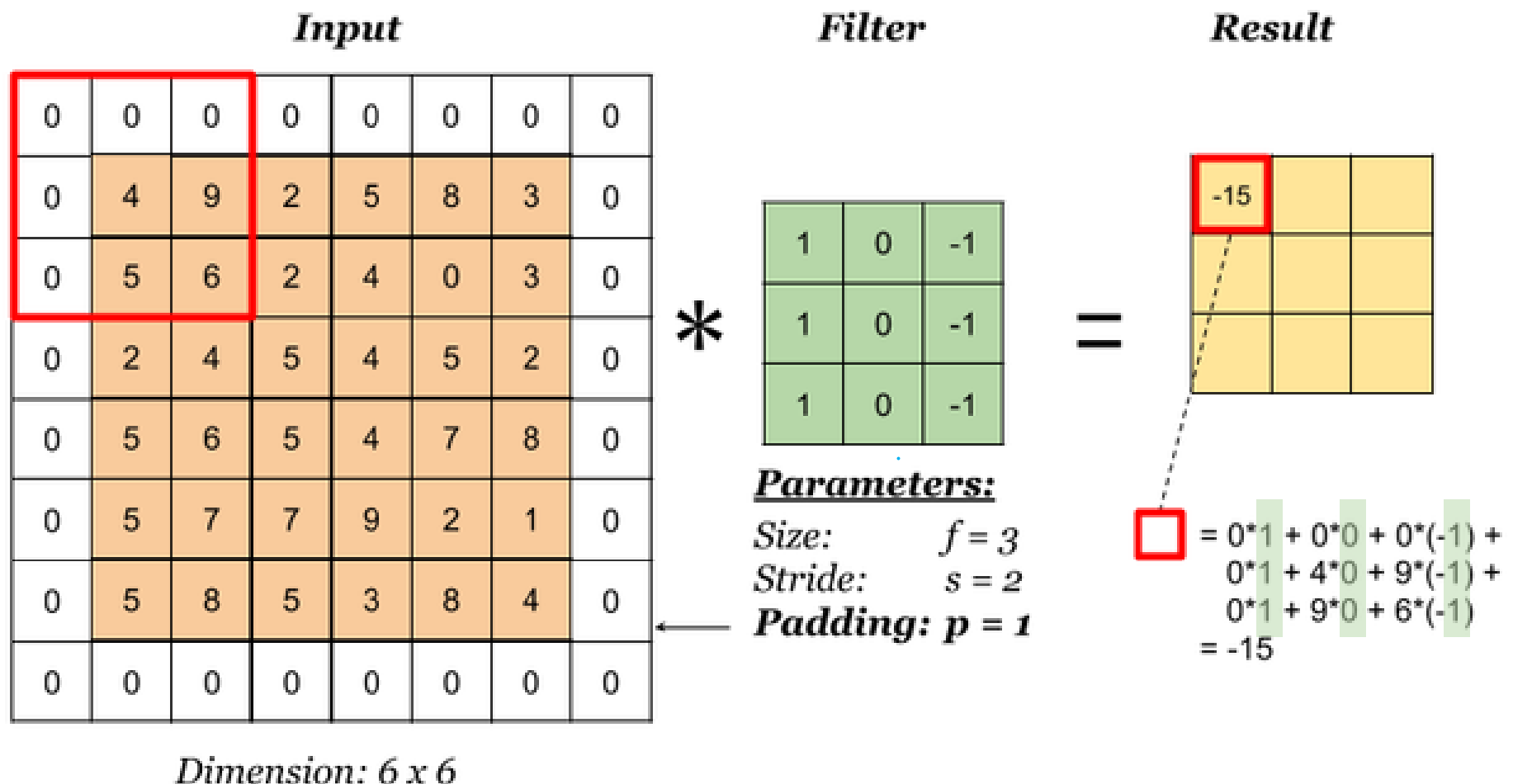
Can add zeros around the border of the image before convolution

Q: Why would we want to add zero padding?

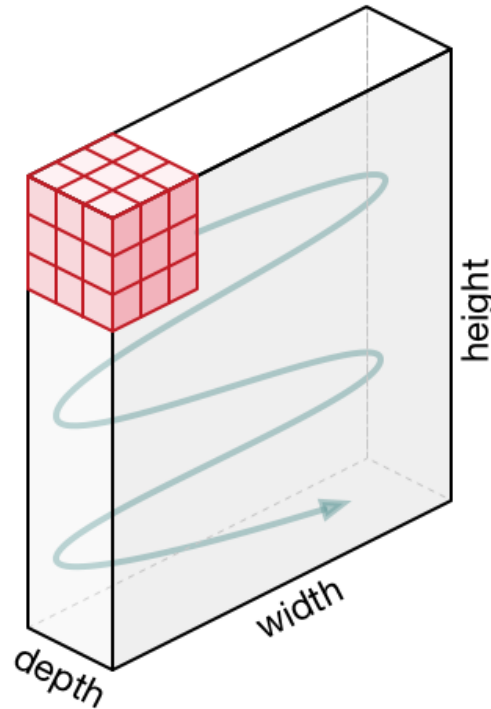
Convolution Calculation: Stride



Convolution Calculation: Padding



What if we have 3 colours?

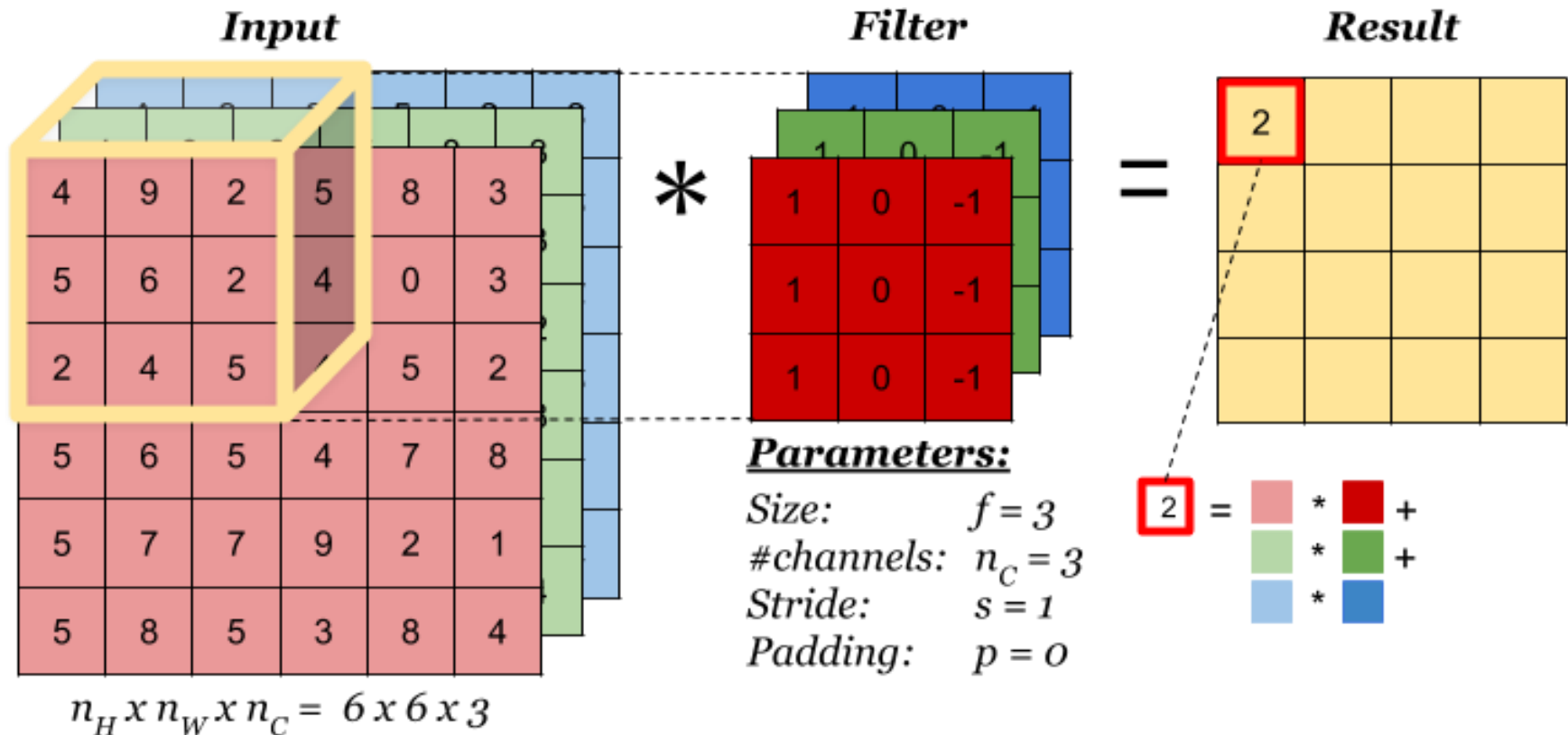


Convolution in RGB

The kernel becomes a 3-dimensional tensor!
In this example, the kernel has size $3 \times 3 \times 3$

RGB Convolution

- Note: Kernel has multiple layers, however the result of the convolution is still one value



Convolutions: RGB Input

Colour input image: $3 \times 28 \times 28$

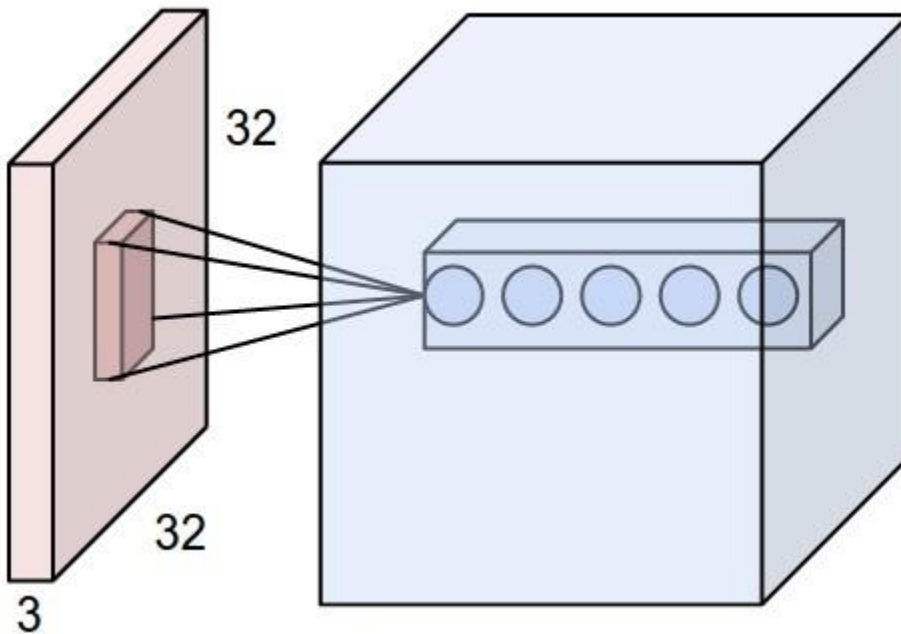
Convolution kernel: $3 \times 3 \times 3$

Questions:

- How many feature maps are in the output layer (higher layer)? What are the dimensions?
- How many trainable weights are there?

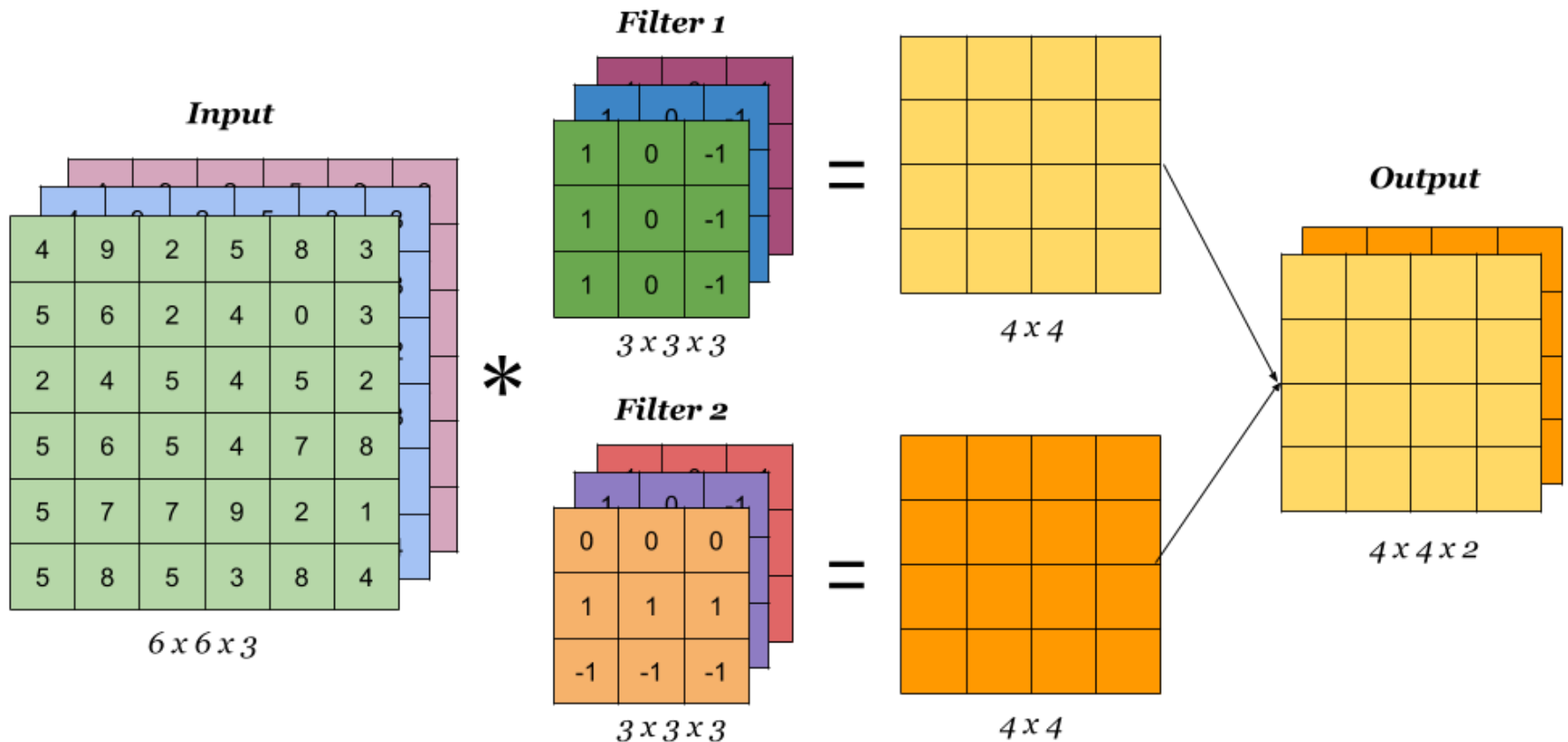
Detecting Multiple Features

Q: What if we want to detect many features of the input? (e.g., both horizontal edges and vertical edges, and maybe even other features?)



Multiple Kernels (Filters)

- The number of outputs (feature maps) is the same as the number of kernels used in the convolution layer.



Example

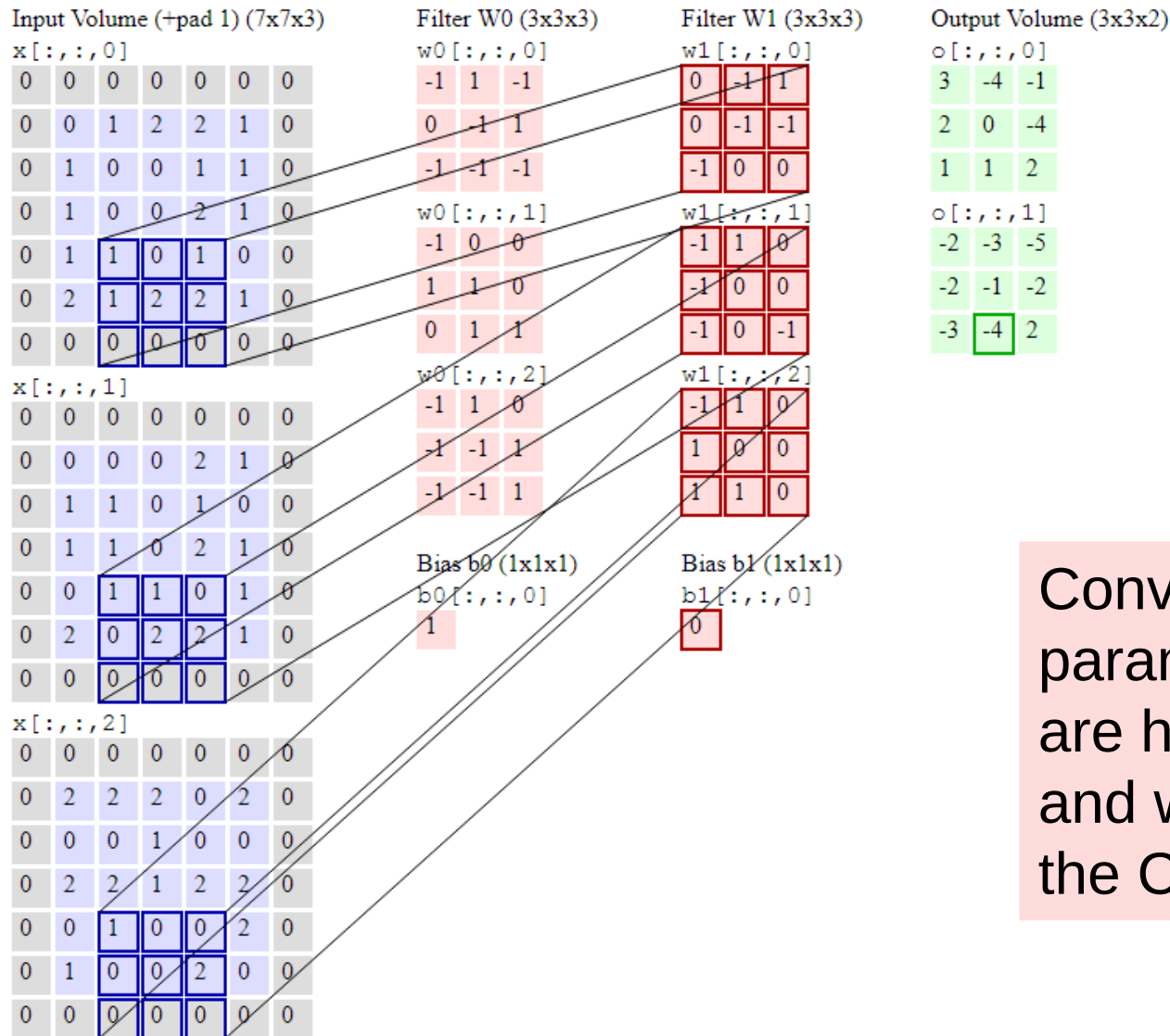
Input features: $5 \times 32 \times 32$

Convolution kernel: $5 \times 3 \times 3 \times 10$

Questions:

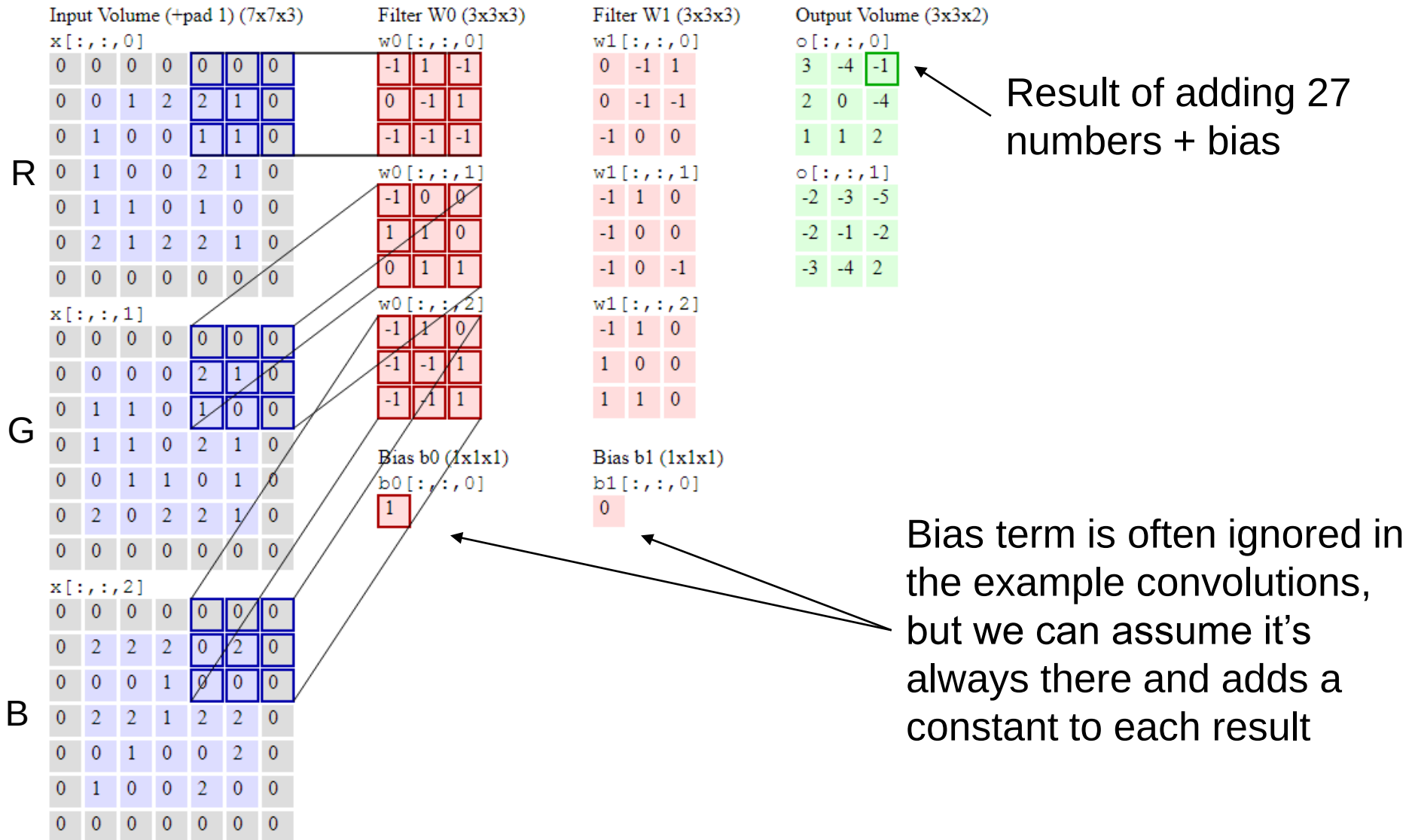
- How many input channels are there?
- How many output channels (feature maps) are there?
- How many trainable weights are there?

Learning Kernel Parameters

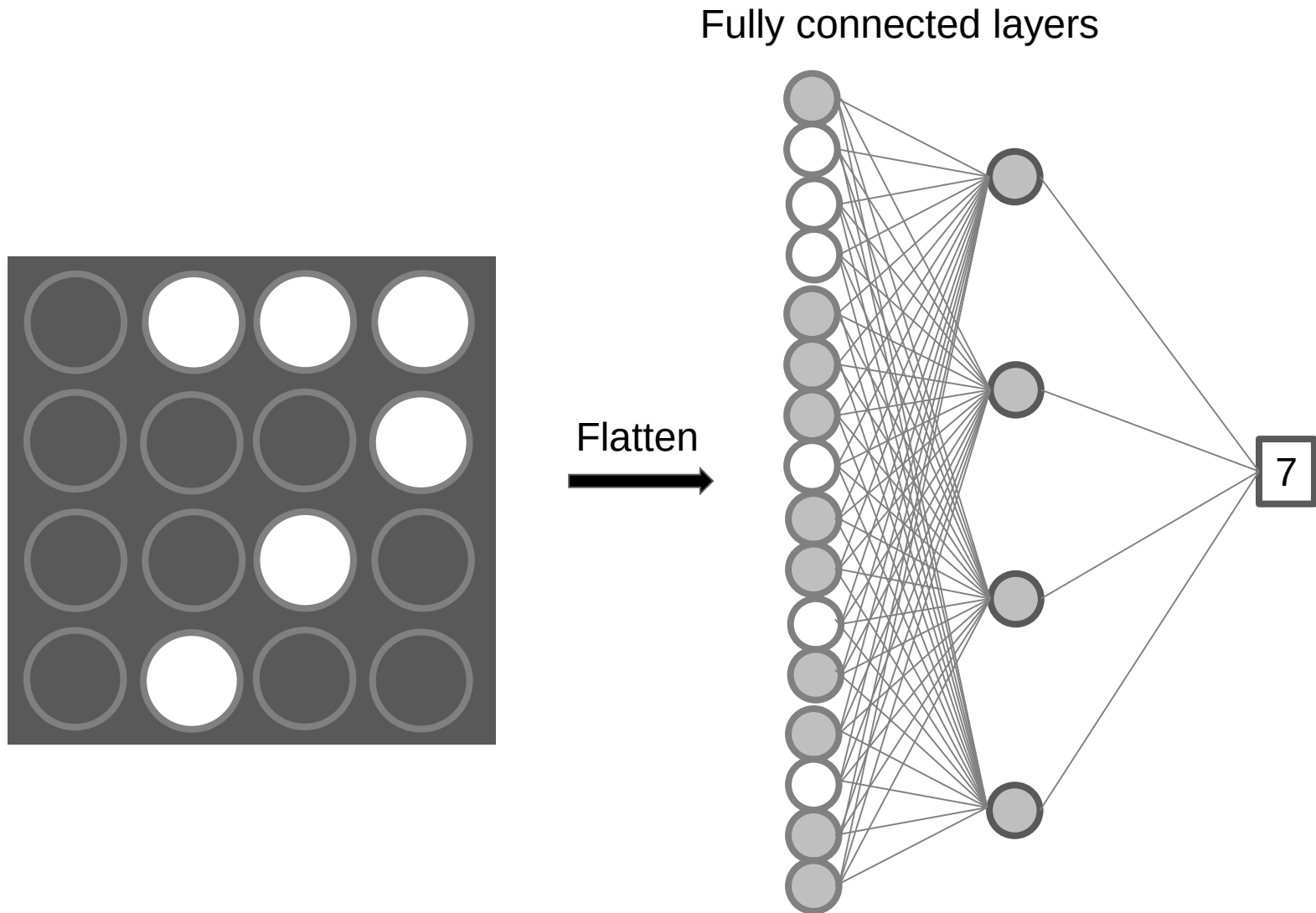


Convolution layer parameters / weights are highlighted in pink and will be learned by the CNN during training

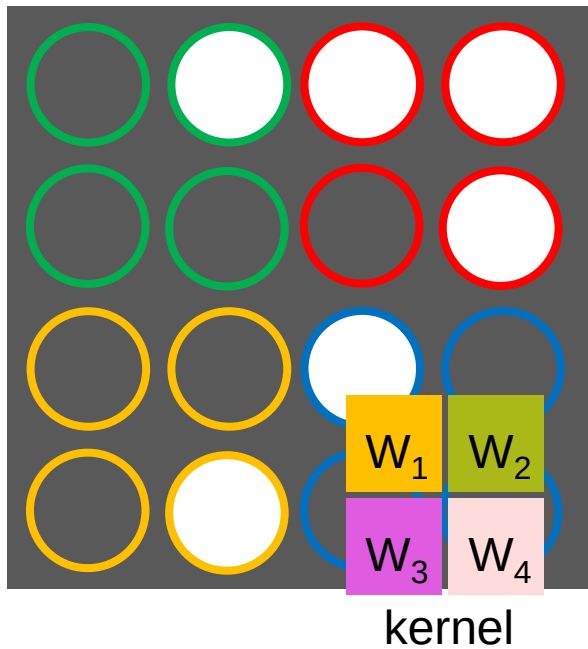
Sample Convolution: Kernel 1



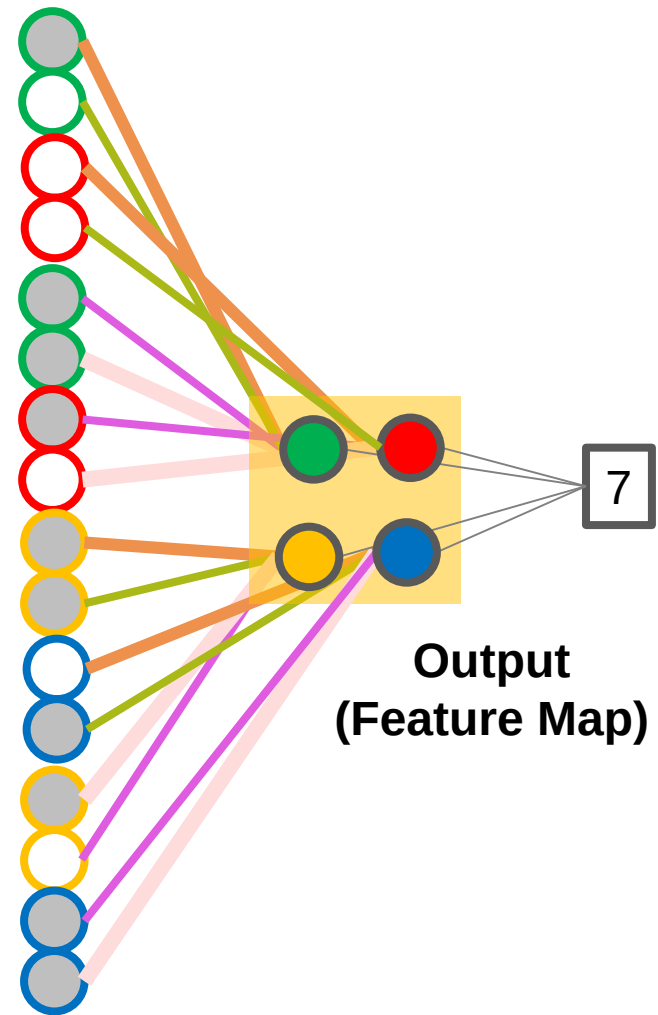
Something to think about...



Linear to Convolutional Layers



convolutional layers



Summary

- Convolutional operations can be achieved using linear layers!
- Convolutional layers constrain the model to take advantage of spatial features.

Week 4 - Announcements

Assignment 2 Released

- Assignment 2 Preparation
 - Pre_2A – CNN
 - Pre_2B – Transfer Learning
 - Ask questions in Support Session or on Piazza
- Assignment 2 – Hand Gesture Recognition
 - Create your own testing dataset (new data)
 - Dataset has errors – find and correct outliers or leave them and see how it affects performance
 - Large part of the evaluation depends on performance on new data
 - Due Tuesday, Oct 7 at 11:00pm

Projects

➤ Team Formations

- Form teams on Quercus
- Piazza “Search for Teammates!” pinned
- Project Lists on Shared Google Spreadsheet
- **1% if done by due date (Tuesday, Sept 30)**

➤ Project Details

- Short project description
- Details and link to dataset(s) you plan on using
- How you will obtain new samples
- **1% if done by due date (Tuesday, Oct 7)**

Final Exam Vote

➤ Results of Final Exam survey

Monday, Dec 1, 8am - 10:30am	12 respondents	29 %
Monday, Dec 1, 1pm - 3:30pm	24 respondents	59 %
Monday, Dec 1, 6pm - 8:30pm	20 respondents	49 %
Wednesday, Dec 3, 8am - 10:30am	15 respondents	37 %
Wednesday, Dec 3, 1pm - 3:30pm	30 respondents	73 %
Wednesday, Dec 3, 7pm - 9:30pm	25 respondents	61 %
Friday, Dec 5, 8am - 10:30am	10 respondents	24 %
Friday, Dec 5, 1pm - 3:30pm	23 respondents	56 %
Friday, Dec 5, 6pm - 8:30pm	24 respondents	59 %

Next Time

Homework

- Review Pre2A Sample Code
- Review Pre2B Sample Code

Q/A Support Session

- Assignment 2
- Project Discussions
- Questions about Sample Code

Lecture Week 5

- CNN Architectures
- Transfer Learning
- Applications